

# NLP group project

March 2, 2024

```
[14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from nltk.corpus import stopwords
import string
import re

df = pd.read_csv('/Users/monicafam/Desktop/amazon_reviews.csv')

[15]: df.drop_duplicates(inplace=True)

[16]: def clean_text(text):
    text = re.sub(' +', ' ', str(text))
    text = re.sub(r'[0-9]', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text

df['clean_narrative'] = df['reviewText'].apply(clean_text)

missing_values = df.isnull().sum()
print('Missing Values:\n', missing_values)

data_clean = dict()
data_clean["narrative"] = np.array(df["reviewText"])
data_clean["clean narrative"] = [""] * len(data_clean["narrative"])
data_clean["vector"] = [""] * len(data_clean["narrative"])
data_clean["score"] = [""] * len(data_clean["narrative"])
data_clean["date"] = np.array(df["reviewTime"])
data_clean["date group"] = [""] * len(data_clean["narrative"])

def clean_white_space(text):
    if pd.isnull(text):
        return ""
    return re.sub(' +', ' ', text)
```

```

def remove_numbers(text):
    if pd.isnull(text):
        return ""
    return re.sub(r'[0-9]', '', text)

def remove_punctuation(text):
    if pd.isnull(text):
        return ""
    return text.translate(str.maketrans('', '', string.punctuation))

data_clean["narrative"] = [clean_white_space(text) for text in data_clean["narrative"]]
data_clean["narrative"] = [remove_numbers(text) for text in data_clean["narrative"]]
data_clean["narrative"] = [remove_punctuation(text) for text in data_clean["narrative"]]

data_clean_df = pd.DataFrame(data_clean)

missing_values_cleaned = data_clean_df.isnull().sum()
print('Missing Values in Cleaned Data:\n', missing_values_cleaned)

data_clean_df.drop_duplicates(inplace=True)

```

Missing Values:

|                      |   |
|----------------------|---|
| Unnamed: 0           | 0 |
| reviewerName         | 1 |
| overall              | 0 |
| reviewText           | 1 |
| reviewTime           | 0 |
| day_diff             | 0 |
| helpful_yes          | 0 |
| helpful_no           | 0 |
| total_vote           | 0 |
| score_pos_neg_diff   | 0 |
| score_average_rating | 0 |
| wilson_lower_bound   | 0 |
| clean_narrative      | 0 |

dtype: int64

Missing Values in Cleaned Data:

|                 |   |
|-----------------|---|
| narrative       | 0 |
| clean narrative | 0 |
| vector          | 0 |
| score           | 0 |
| date            | 0 |
| date group      | 0 |

dtype: int64

```
[17]: data_clean_df['narrative'].fillna('', inplace=True)
```

```
[18]: rows_with_missing_narrative = data_clean_df[data_clean_df['narrative'].isnull()]
print(rows_with_missing_narrative)
```

Empty DataFrame

Columns: [narrative, clean narrative, vector, score, date, date group]

Index: []

```
[19]: def clean_white_space(text):
        return re.sub(' +', ' ', text)

def remove_numbers(text):
    return re.sub(r'[0-9]', '', text)

def remove_punctuation(text):
    return text.translate(str.maketrans('', '', string.punctuation))

POSSIBLE_POS = {"NOUN", "VERB", "ADJ", "ADV", "ADP", "PROPN"}
KEEP_POS = {"NOUN", "VERB", "PROPN"}

def clean_part_of_speech(text):
    pass
```

```
[20]: # Print lengths of arrays in data_clean
for key, value in data_clean.items():
    print(f"Length of {key}: {len(value)}")
```

Length of narrative: 4915

Length of clean narrative: 4915

Length of vector: 4915

Length of score: 4915

Length of date: 4915

Length of date group: 4915

```
[21]: # Check lengths of arrays in data_clean
for key, value in data_clean.items():
    print(f"Length of {key}: {len(value)}")
```

Length of narrative: 4915

Length of clean narrative: 4915

Length of vector: 4915

Length of score: 4915

Length of date: 4915

Length of date group: 4915



```
data_clean["clean narrative"] = [remove_numbers(text) for text in
    ↪data_clean["clean narrative"]]
data_clean["clean narrative"] = [remove_punctuation(text) for text in
    ↪data_clean["clean narrative"]]
```

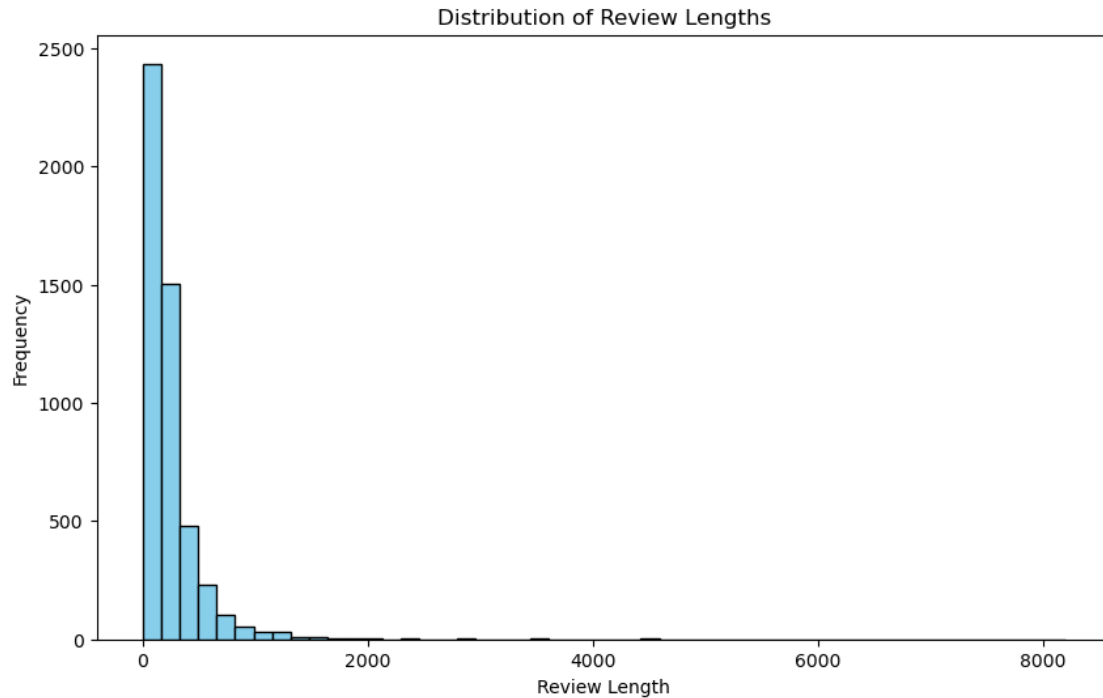
```
-----
NameError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 data_clean["clean narrative"] =
    ↪convert_floats_to_strings(data_clean["clean narrative"])
      2 data_clean["clean narrative"] = [clean_white_space(text) for text in
    ↪data_clean["clean narrative"]]
      3 data_clean["clean narrative"] = [remove_numbers(text) for text in
    ↪data_clean["clean narrative"]]

NameError: name 'convert_floats_to_strings' is not defined
```

```
[26]: data_clean['reviewLength'] = [len(text) for text in data_clean['narrative']]
```

```
[27]: import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.hist(data_clean['reviewLength'], bins=50, color='skyblue',
    ↪edgecolor='black')
plt.title('Distribution of Review Lengths')
plt.xlabel('Review Length')
plt.ylabel('Frequency')
plt.show()
```



```
[28]: import pandas as pd

data_clean = pd.DataFrame({'narrative': data_clean['narrative']})

data_clean['reviewLength'] = [len(text) for text in data_clean['narrative']]

zero_length_reviews = data_clean[data_clean['reviewLength'] == 0].
    ↪reset_index(drop=True)
print(zero_length_reviews)
```

```
   narrative  reviewLength
0          0             0
```

```
[29]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.countplot(x='overall', data=df, palette='viridis')
plt.title('Distribution of Overall Ratings')
plt.xlabel('Overall Rating')
plt.ylabel('Number of Reviews')
plt.show()
```

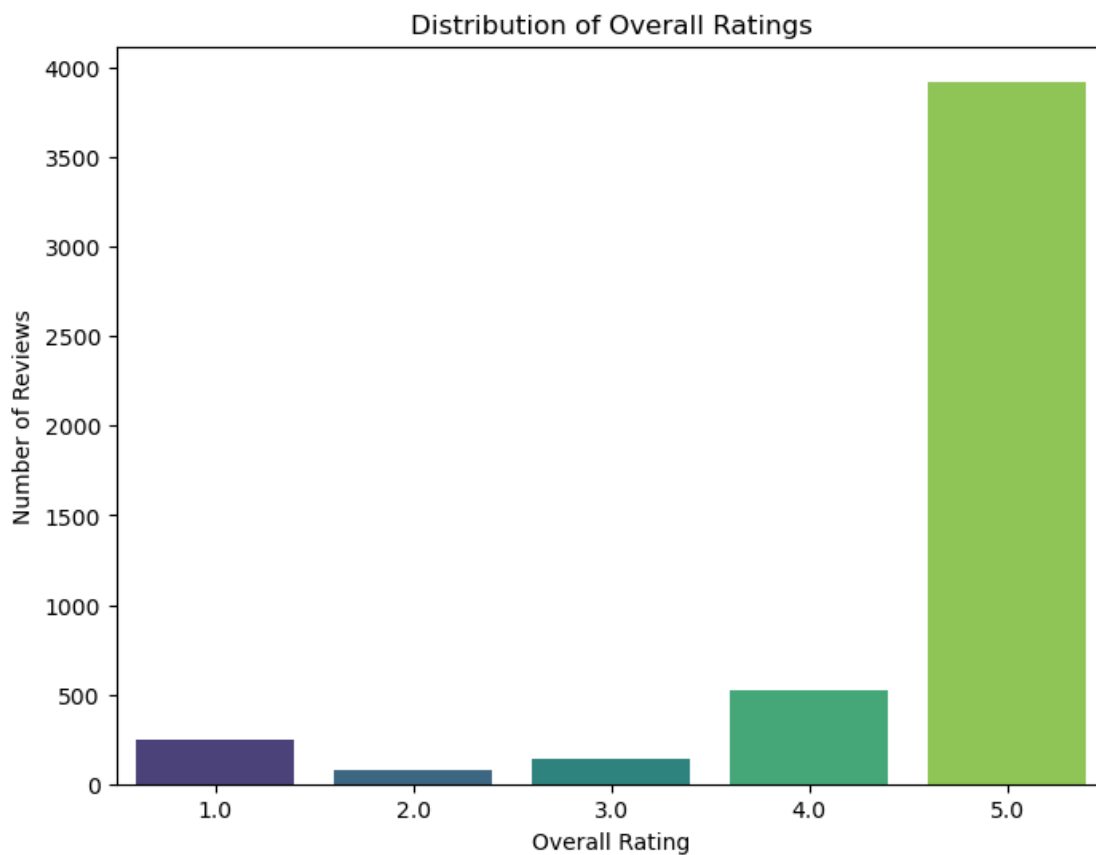
```
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
```

deprecated and will be removed in a future version. Use `isinstance(dtype, CategoricalDtype)` instead

```
if pd.api.types.is_categorical_dtype(vector):  
/Users/monicafam/anaconda3/lib/python3.11/site-  
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is  
deprecated and will be removed in a future version. Use isinstance(dtype,  
CategoricalDtype) instead
```

```
if pd.api.types.is_categorical_dtype(vector):  
/Users/monicafam/anaconda3/lib/python3.11/site-  
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is  
deprecated and will be removed in a future version. Use isinstance(dtype,  
CategoricalDtype) instead
```

```
if pd.api.types.is_categorical_dtype(vector):
```

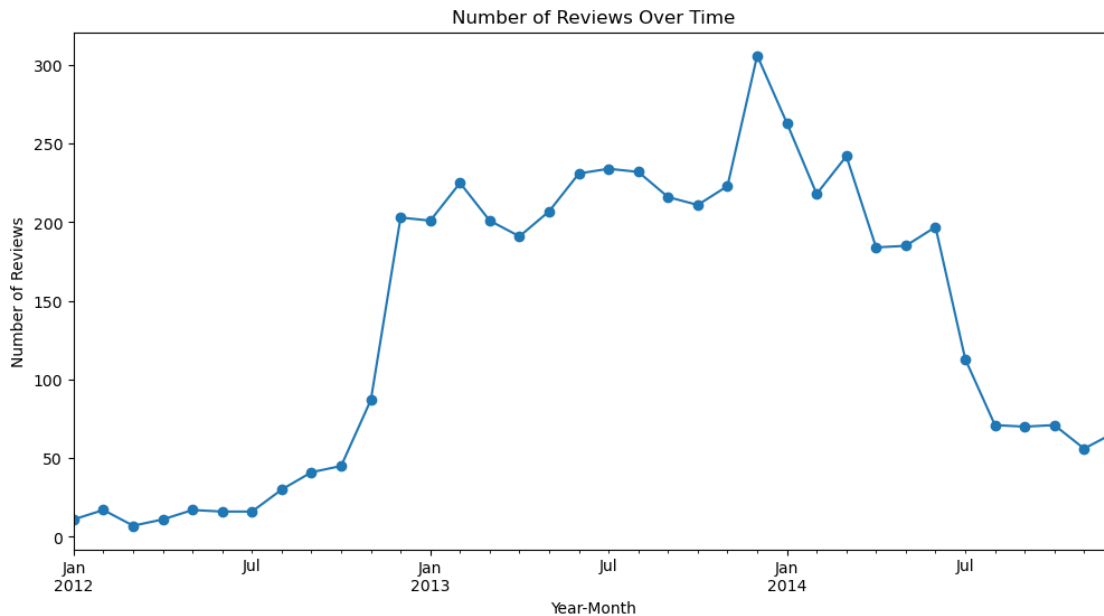


```
[30]: #Timeseries Analysis
```

```
df['reviewTime'] = pd.to_datetime(df['reviewTime'])
```

```
df['year_month'] = df['reviewTime'].dt.to_period('M')
```

```
plt.figure(figsize=(12, 6))
df['year_month'].value_counts().sort_index().plot(kind='line', marker='o')
plt.title('Number of Reviews Over Time')
plt.xlabel('Year-Month')
plt.ylabel('Number of Reviews')
plt.show()
```



```
[31]: plt.figure(figsize=(8, 6))
sns.countplot(x='helpful_yes', data=df, palette='coolwarm', label='Yes')
sns.countplot(x='helpful_no', data=df, palette='Oranges', label='No')
plt.title('Distribution of Helpful Votes')
plt.xlabel('Number of Votes')
plt.ylabel('Number of Reviews')
plt.legend(title='Helpfulness')
plt.show()
```

/Users/monicafam/anaconda3/lib/python3.11/site-packages/seaborn/\_oldcore.py:1498: FutureWarning: is\_categorical\_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead

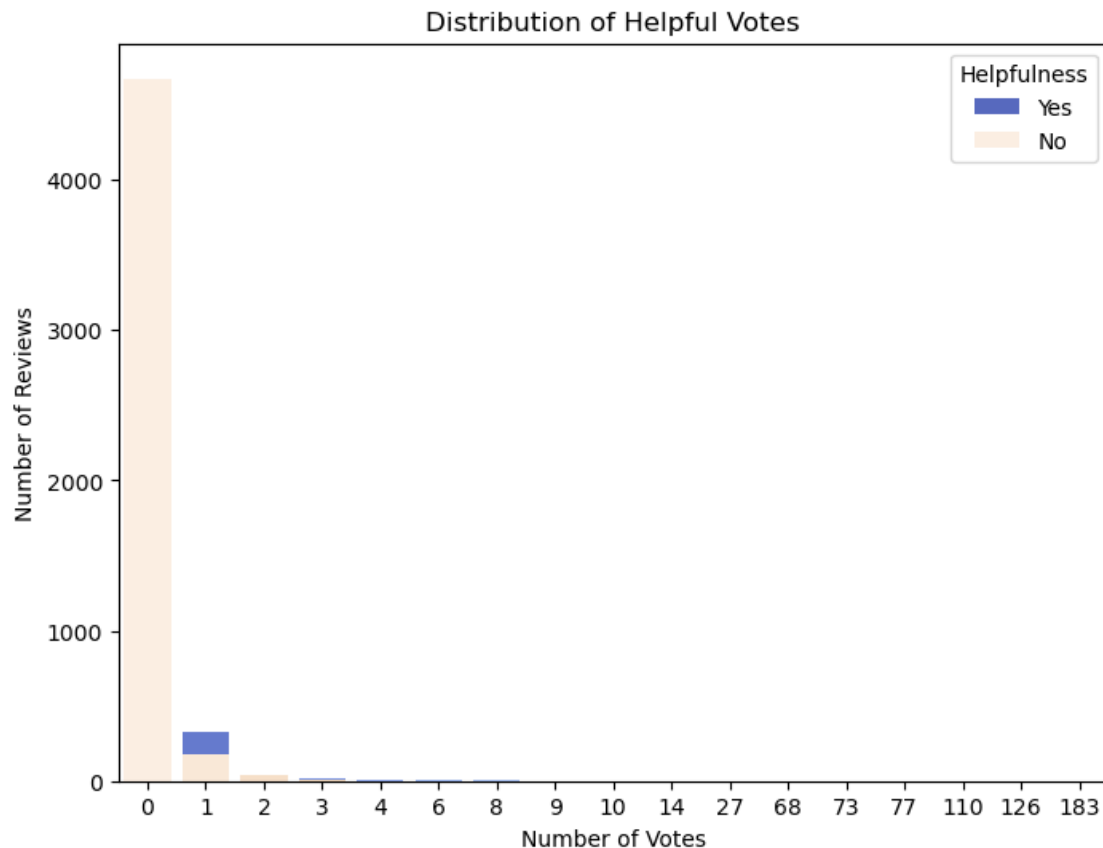
```
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
```



```

if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead

```



```
[32]: #Bar Timeseries
df['reviewTime'] = pd.to_datetime(df['reviewTime'])

df['year_month'] = df['reviewTime'].dt.to_period('M')

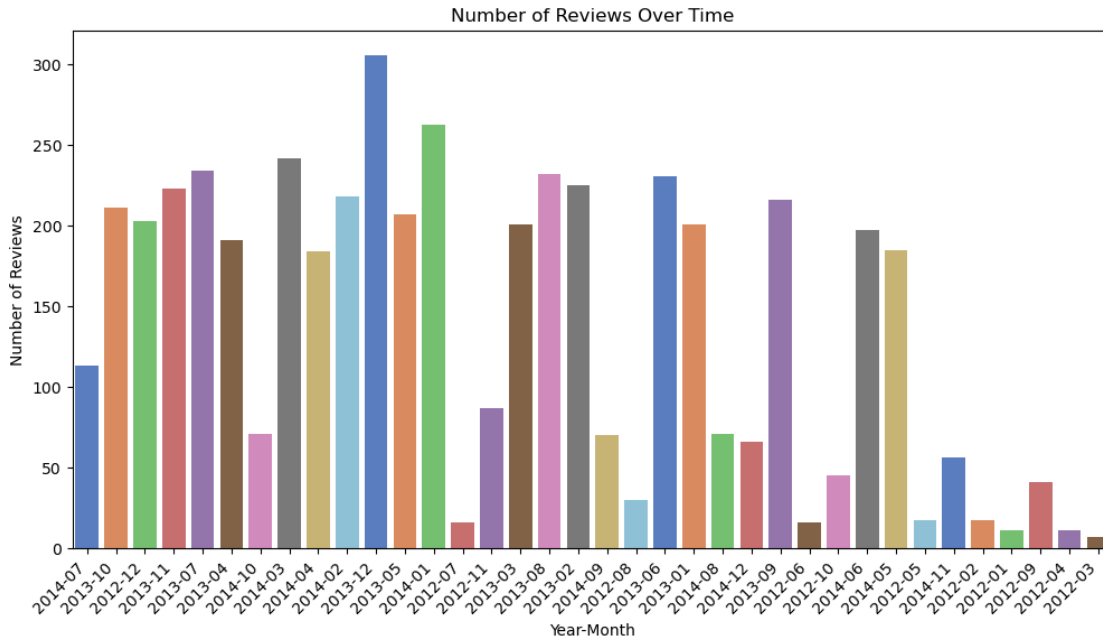
plt.figure(figsize=(12, 6))
sns.countplot(x='year_month', data=df, palette='muted')
plt.title('Number of Reviews Over Time')
plt.xlabel('Year-Month')
plt.ylabel('Number of Reviews')
plt.xticks(rotation=45, ha='right')
plt.show()
```

```
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
```

```
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
```

```
if pd.api.types.is_categorical_dtype(vector):
/Users/monicafam/anaconda3/lib/python3.11/site-
packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is
deprecated and will be removed in a future version. Use isinstance(dtype,
CategoricalDtype) instead
```

```
if pd.api.types.is_categorical_dtype(vector):
```



[33]: *#text analysis*

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

corpus = data_clean_df['narrative']

vectorizer = CountVectorizer(max_features=1000, stop_words='english')
X = vectorizer.fit_transform(corpus)

lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(X)

feature_names = vectorizer.get_feature_names_out()
for topic_idx, topic in enumerate(lda.components_):
    top_words_idx = topic.argsort()[:-10 - 1:-1]
    top_words = [feature_names[i] for i in top_words_idx]
    print(f"Topic #{topic_idx + 1}: {' '.join(top_words)}")
```

Topic #1: card, phone, sd, gb, memory, adapter, works, micro, tablet, great

Topic #2: card, gb, sandisk, cards, phone, just, read, work, format, class

Topic #3: sandisk, card, price, good, great, works, memory, product, speed, cards

Topic #4: galaxy, works, samsung, great, music, storage, note, tablet, use, gb

Topic #5: camera, card, video, gopro, works, use, great, memory, fast, hd

```
[37]: !python -m spacy download en_core_web_sm
```

Collecting en-core-web-sm==3.7.1

Downloading [https://github.com/explosion/spacy-models/releases/download/en\\_core\\_web\\_sm-3.7.1/en\\_core\\_web\\_sm-3.7.1-py3-none-any.whl](https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.7.1/en_core_web_sm-3.7.1-py3-none-any.whl) (12.8 MB)

12.8/12.8 MB

48.8 MB/s eta 0:00:0000:0100:01

Requirement already satisfied: spacy<3.8.0,>=3.7.2 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from en-core-web-sm==3.7.1) (3.7.4)

Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (3.0.12)

Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.0.5)

Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.0.10)

Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.0.8)

Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (3.0.9)

Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (8.2.3)

Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.1.2)

Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.4.8)

Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.0.10)

Requirement already satisfied: weasel<0.4.0,>=0.1.0 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (0.3.4)

Requirement already satisfied: typer<0.10.0,>=0.3.0 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (0.9.0)

Requirement already satisfied: smart-open<7.0.0,>=5.2.1 in /Users/monicafam/anaconda3/lib/python3.11/site-packages (from spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (5.2.1)

Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in

/Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (4.65.0)  
 Requirement already satisfied: requests<3.0.0,>=2.13.0 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.31.0)  
 Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.10.8)  
 Requirement already satisfied: jinja2 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (3.1.2)  
 Requirement already satisfied: setuptools in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (69.1.1)  
 Requirement already satisfied: packaging>=20.0 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (23.1)  
 Requirement already satisfied: langcodes<4.0.0,>=3.2.0 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (3.3.0)  
 Requirement already satisfied: numpy>=1.19.0 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.24.3)  
 Requirement already satisfied: typing-extensions>=4.2.0 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy<3.8.0,>=3.7.2->en-core-web-  
 sm==3.7.1) (4.7.1)  
 Requirement already satisfied: charset-normalizer<4,>=2 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 requests<3.0.0,>=2.13.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.0.4)  
 Requirement already satisfied: idna<4,>=2.5 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 requests<3.0.0,>=2.13.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (3.4)  
 Requirement already satisfied: urllib3<3,>=1.21.1 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 requests<3.0.0,>=2.13.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (1.26.16)  
 Requirement already satisfied: certifi>=2017.4.17 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 requests<3.0.0,>=2.13.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2023.7.22)  
 Requirement already satisfied: blis<0.8.0,>=0.7.8 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 thinc<8.3.0,>=8.2.2->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (0.7.11)  
 Requirement already satisfied: confection<1.0.0,>=0.0.1 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 thinc<8.3.0,>=8.2.2->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (0.1.4)  
 Requirement already satisfied: click<9.0.0,>=7.1.1 in  
 /Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
 typer<0.10.0,>=0.3.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (8.0.4)

Requirement already satisfied: cloudpathlib<0.17.0,>=0.7.0 in  
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
weasel<0.4.0,>=0.1.0->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (0.16.0)  
Requirement already satisfied: MarkupSafe>=2.0 in  
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from  
jinja2->spacy<3.8.0,>=3.7.2->en-core-web-sm==3.7.1) (2.1.1)  
Download and installation successful  
You can now load the package via spacy.load('en\_core\_web\_sm')

```
[38]: import nltk
import spacy
import pandas as pd
import numpy as np
import sklearn
import plotly
import re
import string
import os
import multiprocessing as mp
import spacy
import spacy
nlp = spacy.load('en_core_web_sm')
```

```
[39]: new_dir = "/Users/monicafam/Desktop/"
os.chdir(new_dir)

current_dir = os.getcwd()
print("Current directory:", current_dir)
```

Current directory: /Users/monicafam/Desktop

```
[41]: from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
import string

def remove_stop_words(text):
    stop_words = set(stopwords.words('english'))
    words = word_tokenize(text)
    filtered_words = [word for word in words if word.lower() not in stop_words]
    return ' '.join(filtered_words)

def remove_numbers(text):

    pass
```

```

def remove_punctuation(text):
    pass

def clean_part_of_speech(text):
    pass

def remove_X(text):
    pass

def clean_white_space(text):
    pass

```

```

[61]: import pandas as pd
from sklearn.model_selection import train_test_split

data = {
    'reviewText': [
        "This is a positive review.",
        "This is another positive review.",
        "This is a negative review.",
        "This is another negative review."
    ],
    'sentiment': [1, 1, 0, 0]
}

df_raw = pd.DataFrame(data)

def run_clean_job(ls):
    clean_text = list()
    for narrative in ls:
        if narrative is not None and len(narrative) > 0:
            narrative = narrative.strip()
            clean_text.append(narrative)
    return clean_text

df_raw["clean_narrative"] = run_clean_job(df_raw["reviewText"])

df_clean = pd.DataFrame({
    'clean_narrative': df_raw["clean_narrative"],
    'sentiment': df_raw["sentiment"]
})

X_train, X_test, y_train, y_test = train_test_split(

```

```
df_clean['clean_narrative'],
df_clean['sentiment'],
test_size=0.2,
random_state=42
)
```

```
[62]: df = pd.read_csv('amazon_reviews.csv')
n_rows = 100000
df_raw = pd.read_csv("amazon_reviews.csv").head(n_rows)
df_raw.head(10)
```

| [62]: | Unnamed: 0 | reviewerName                                     | overall | \ |
|-------|------------|--------------------------------------------------|---------|---|
| 0     | 0          | NaN                                              | 4.0     |   |
| 1     | 1          | Omie                                             | 5.0     |   |
| 2     | 2          | 1K3                                              | 4.0     |   |
| 3     | 3          | 1m2                                              | 5.0     |   |
| 4     | 4          | 2&1/2Men                                         | 5.0     |   |
| 5     | 5          | 2Cents!                                          | 5.0     |   |
| 6     | 6          | 2K1Toaster                                       | 5.0     |   |
| 7     | 7          | 35-year Technology Consumer "8-tracks to 802.11" | 5.0     |   |
| 8     | 8          | 4evryoung                                        | 5.0     |   |
| 9     | 9          | 53rdcard                                         | 5.0     |   |

|   | reviewText                                        | reviewTime | day_diff | \ |
|---|---------------------------------------------------|------------|----------|---|
| 0 | No issues.                                        | 2014-07-23 | 138      |   |
| 1 | Purchased this for my device, it worked as adv... | 2013-10-25 | 409      |   |
| 2 | it works as expected. I should have sprung for... | 2012-12-23 | 715      |   |
| 3 | This think has worked out great.Had a diff. br... | 2013-11-21 | 382      |   |
| 4 | Bought it with Retail Packaging, arrived legit... | 2013-07-13 | 513      |   |
| 5 | It's mini storage. It doesn't do anything els...  | 2013-04-29 | 588      |   |
| 6 | I have it in my phone and it never skips a bea... | 2013-10-19 | 415      |   |
| 7 | It's hard to believe how affordable digital ha... | 2014-10-07 | 62       |   |
| 8 | Works in a HTC Rezound. Was running short of ...  | 2014-03-24 | 259      |   |
| 9 | in my galaxy s4, super fast card, and am total... | 2013-11-10 | 393      |   |

|   | helpful_yes | helpful_no | total_vote | score_pos_neg_diff | \ |
|---|-------------|------------|------------|--------------------|---|
| 0 | 0           | 0          | 0          | 0                  |   |
| 1 | 0           | 0          | 0          | 0                  |   |
| 2 | 0           | 0          | 0          | 0                  |   |
| 3 | 0           | 0          | 0          | 0                  |   |
| 4 | 0           | 0          | 0          | 0                  |   |
| 5 | 0           | 0          | 0          | 0                  |   |
| 6 | 0           | 0          | 0          | 0                  |   |
| 7 | 0           | 0          | 0          | 0                  |   |
| 8 | 1           | 0          | 1          | 1                  |   |
| 9 | 0           | 0          | 0          | 0                  |   |



|   | score_average_rating | wilson_lower_bound |
|---|----------------------|--------------------|
| 0 | 0.0                  | 0.000000           |
| 1 | 0.0                  | 0.000000           |
| 2 | 0.0                  | 0.000000           |
| 3 | 0.0                  | 0.000000           |
| 4 | 0.0                  | 0.000000           |
| 5 | 0.0                  | 0.000000           |
| 6 | 0.0                  | 0.000000           |
| 7 | 0.0                  | 0.000000           |
| 8 | 1.0                  | 0.206549           |
| 9 | 0.0                  | 0.000000           |

```
[63]: data_clean = dict()
```

```
[64]: print(data_clean.keys())
```

```
dict_keys([])
```

```
[67]: # identify commentary that matches
data_clean["narrative"] = np.array(df_raw["reviewText"])
data_clean["clean narrative"] = list()
data_clean["vector"] = list()
data_clean["score"] = list()
# create groups to track frequency
data_clean["date"] = np.array(df_raw["reviewTime"])
data_clean["date group"] = list()
```

```
[68]: data_clean["narrative"][1]
```

```
[68]: 'Purchased this for my device, it worked as advertised. You can never have too
much phone memory, since I download a lot of stuff this was a no brainer for
me.'
```

```
[ ]:
```

```
[69]: def clean_white_space(text):
        return re.sub(' +', ' ', text)

def remove_numbers(text):
    return re.sub(r'[0-9]', '', text)

def remove_punctuation(text):
    return text.translate(str.maketrans('', '', string.punctuation))

POSSIBLE_POS = {"NOUN", "VERB", "ADJ", "ADV", "ADP", "PROPN"}
KEEP_POS = {"NOUN", "VERB", "PROPN"}
def clean_part_of_speech(text):
    keep = list()
```

```

tokens = nlp(text)
for token in tokens:
    if token.pos_ in KEEP_POS:
        keep.append(token.text)
return " ".join(keep)

stopwords = nlp.Defaults.stop_words
def remove_stop_words(text):
    keep = list()
    tokens = nlp(text)
    for token in tokens:
        if token.text.lower().strip() not in stopwords:
            keep.append(token.text)
    return " ".join(keep)

def remove_X(text):
    return text.replace("X", "")

# clean
def run_clean_job(ls):
    clean_text = list()
    for narrative in ls:
        narrative = narrative.strip()
        narrative = remove_numbers(narrative)
        narrative = remove_punctuation(narrative)
        narrative = remove_stop_words(narrative)
        narrative = clean_part_of_speech(narrative)
        narrative = remove_X(narrative)
        narrative = clean_white_space(narrative)
        clean_text.append(narrative)
    return clean_text

```

```

[70]: def convert_floats_to_strings(text_array):
    # Convert each element to string and process
    for i, text in np.ndenumerate(text_array):
        # Convert bytes-like object to string
        text_str = text.decode('utf-8') if isinstance(text, bytes) else
↳ str(text)

        # Regular expression pattern to match floats
        float_pattern = r'\b\d+\.\d+\b'

        # Find all floats in the text
        floats = re.findall(float_pattern, text_str)

```

```

    # Replace each float with its string representation
    for float_str in floats:
        text_str = text_str.replace(float_str, str(float_str))

    # Assign the modified text back to the array
    text_array[i] = text_str

    return text_array

```

```

[71]: # Convert bytes to strings and convert floats to strings
      convert_floats_to_strings(data_clean["narrative"])
      run_clean_job(data_clean["narrative"])
      print("done")

```

done

```

[72]: data_clean["narrative"][1]

```

```

[72]: 'Purchased this for my device, it worked as advertised. You can never have too
      much phone memory, since I download a lot of stuff this was a no brainer for
      me.'

```

```

[73]: from nltk.sentiment.vader import SentimentIntensityAnalyzer
      import nltk
      nltk.download('vader_lexicon')

```

```

[nltk_data] Downloading package vader_lexicon to
[nltk_data] /Users/monicafam/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

```

```

[73]: True

```

```

[74]: # Initialize VADER sentiment analyzer
      sid = SentimentIntensityAnalyzer()

      # Analyze sentiment of a sentence
      sentences = data_clean["narrative"]
      sentiment_scores = []

      for i, sentence in enumerate(sentences):
          scores = sid.polarity_scores(sentence)
          sentiment_scores.append((i, scores['compound']))

      sentiment_scores.sort(key=lambda x: x[1], reverse=True)

      print("Top 10 ranked sentences:")

```

```

for rank, (index, _) in enumerate(sentiment_scores[:10], start=1):
    print(f"Sentence at index {index} is ranked {rank}.")

print("\nBottom 10 ranked sentences:")
for rank, (index, _) in enumerate(sentiment_scores[-10:], start=1):
    print(f"Sentence at index {index} is ranked {len(sentiment_scores) - rank + 1}.")

```

Top 10 ranked sentences:

```

Sentence at index 4212 is ranked 1.
Sentence at index 2031 is ranked 2.
Sentence at index 2576 is ranked 3.
Sentence at index 2802 is ranked 4.
Sentence at index 2881 is ranked 5.
Sentence at index 4634 is ranked 6.
Sentence at index 1177 is ranked 7.
Sentence at index 4423 is ranked 8.
Sentence at index 3449 is ranked 9.
Sentence at index 2993 is ranked 10.

```

Bottom 10 ranked sentences:

```

Sentence at index 4092 is ranked 4915.
Sentence at index 4034 is ranked 4914.
Sentence at index 1176 is ranked 4913.
Sentence at index 4587 is ranked 4912.
Sentence at index 4368 is ranked 4911.
Sentence at index 1760 is ranked 4910.
Sentence at index 2889 is ranked 4909.
Sentence at index 3589 is ranked 4908.
Sentence at index 1904 is ranked 4907.
Sentence at index 615 is ranked 4906.

```

```

[75]: sentence = data_clean["narrative"][4212]
      scores = sid.polarity_scores(sentence)
      print(scores)

```

```

{'neg': 0.045, 'neu': 0.814, 'pos': 0.141, 'compound': 0.9994}

```

```

[76]: def label_row(value):
      if value == 5 or value == 4:
          return 1
      elif value == 2 or value == 1:
          return -1
      else:
          return 0

      # Apply the function to create a new column with labels

```

```
df_raw['sentiment'] = df_raw['overall'].apply(lambda x: label_row(x))
```

```
[77]: df_raw
```

```
[77]:
```

|      | Unnamed: 0 | reviewerName | overall | \   |
|------|------------|--------------|---------|-----|
| 0    | 0          | NaN          | 4.0     |     |
| 1    | 1          | Omie         | 5.0     |     |
| 2    | 2          | 1K3          | 4.0     |     |
| 3    | 3          | 1m2          | 5.0     |     |
| 4    | 4          | 2&1/2Men     | 5.0     |     |
| ...  | ...        | ...          | ...     | ... |
| 4910 | 4910       | ZM "J"       | 1.0     |     |
| 4911 | 4911       | Zo           | 5.0     |     |
| 4912 | 4912       | Z S Liske    | 5.0     |     |
| 4913 | 4913       | Z Taylor     | 5.0     |     |
| 4914 | 4914       | Zza          | 5.0     |     |

|      | reviewText                                        | reviewTime | day_diff | \   |
|------|---------------------------------------------------|------------|----------|-----|
| 0    | No issues.                                        | 2014-07-23 | 138      |     |
| 1    | Purchased this for my device, it worked as adv... | 2013-10-25 | 409      |     |
| 2    | it works as expected. I should have sprung for... | 2012-12-23 | 715      |     |
| 3    | This think has worked out great.Had a diff. br... | 2013-11-21 | 382      |     |
| 4    | Bought it with Retail Packaging, arrived legit... | 2013-07-13 | 513      |     |
| ...  | ...                                               | ...        | ...      | ... |
| 4910 | I bought this Sandisk 16GB Class 10 to use wit... | 2013-07-23 | 503      |     |
| 4911 | Used this for extending the capabilities of my... | 2013-08-22 | 473      |     |
| 4912 | Great card that is very fast and reliable. It ... | 2014-03-31 | 252      |     |
| 4913 | Good amount of space for the stuff I want to d... | 2013-09-16 | 448      |     |
| 4914 | I've heard bad things about this 64gb Micro SD... | 2014-02-01 | 310      |     |

|      | helpful_yes | helpful_no | total_vote | score_pos_neg_diff | \   |
|------|-------------|------------|------------|--------------------|-----|
| 0    | 0           | 0          | 0          | 0                  |     |
| 1    | 0           | 0          | 0          | 0                  |     |
| 2    | 0           | 0          | 0          | 0                  |     |
| 3    | 0           | 0          | 0          | 0                  |     |
| 4    | 0           | 0          | 0          | 0                  |     |
| ...  | ...         | ...        | ...        | ...                | ... |
| 4910 | 0           | 0          | 0          | 0                  |     |
| 4911 | 0           | 0          | 0          | 0                  |     |
| 4912 | 0           | 0          | 0          | 0                  |     |
| 4913 | 0           | 0          | 0          | 0                  |     |
| 4914 | 0           | 0          | 0          | 0                  |     |

|   | score_average_rating | wilson_lower_bound | sentiment |
|---|----------------------|--------------------|-----------|
| 0 | 0.0                  | 0.0                | 1         |
| 1 | 0.0                  | 0.0                | 1         |
| 2 | 0.0                  | 0.0                | 1         |

|      |     |     |     |
|------|-----|-----|-----|
| 3    | 0.0 | 0.0 | 1   |
| 4    | 0.0 | 0.0 | 1   |
| ...  | ... | ... | ... |
| 4910 | 0.0 | 0.0 | -1  |
| 4911 | 0.0 | 0.0 | 1   |
| 4912 | 0.0 | 0.0 | 1   |
| 4913 | 0.0 | 0.0 | 1   |
| 4914 | 0.0 | 0.0 | 1   |

[4915 rows x 13 columns]

```
[78]: def label_row(value):
        if value == 5 or value == 4:
            return 1
        elif value == 2 or value == 1:
            return -1
        else:
            return 0

        # Apply the function to create a new column with labels
        df_raw['sentiment'] = df_raw['overall'].apply(lambda x: label_row(x))
```

```
[79]: positvedf = filtered_df = df_raw[df_raw['sentiment'] == 1]
        neutraldf = filtered_df = df_raw[df_raw['sentiment'] == 0]
        negativedf = filtered_df = df_raw[df_raw['sentiment'] == -1]
```

```
[80]: print(len(positvedf))
        print(len(neutraldf))
        print(len(negativedf))
```

4449  
142  
324

```
[81]: from sklearn.model_selection import train_test_split
```

```
[82]: x_train, x_test, y_train, y_test = train_test_split(df_raw["reviewText"],
        ↪df_raw["sentiment"], test_size=0.3, random_state=42)
```

```
[83]: x_train
```

```
[83]: 2217    Popped in my Galaxy S5. Works.Disk came with a...
        3405    My new Android phone (T-Mobile upgrade) has a ...
        2770    I don't know why but I just needed some space...
        286     What more can you ask for ? No errors ever de...
        2245    Very fast reading and writing for my samsung g...
        ...
        4426    Use with my GoPro Hero 3 & class 10 is a must...
```

```
466      Fast shipping & delivered as described.Phone d...
3092     Bought two, one for me and one for my husband...
3772     This 32GB card is fast.  Great for storing vid...
860      Previously purchased two ADATA microsd cards o...
Name: reviewText, Length: 3440, dtype: object
```

```
[84]: y_train
```

```
[84]: 2217    1
      3405    1
      2770    1
      286    1
      2245    1
      ..
      4426    1
      466    1
      3092    1
      3772    1
      860    1
Name: sentiment, Length: 3440, dtype: int64
```

```
[85]: print("Shape of training data (X_train):", x_train.shape)
      print("Shape of testing data (X_test):", x_test.shape)
      print("Shape of training labels (y_train):", y_train.shape)
      print("Shape of testing labels (y_test):", y_test.shape)
```

```
Shape of training data (X_train): (3440,)
Shape of testing data (X_test): (1475,)
Shape of training labels (y_train): (3440,)
Shape of testing labels (y_test): (1475,)
```

```
[86]: from sklearn.model_selection import train_test_split
      from sklearn.feature_extraction.text import CountVectorizer
      from sklearn.naive_bayes import MultinomialNB
      from sklearn.metrics import accuracy_score, classification_report, \
      ↪confusion_matrix
```

```
[94]: import pandas as pd
      from sklearn.model_selection import StratifiedKFold

data = {
    'reviewText': [
        "This is a positive review.",
        "This is another positive review.",
        "This is a negative review.",
        "This is another negative review."
    ],
    'sentiment': [1, 1, 0, 0]
```

```

}

df_raw = pd.DataFrame(data)

def run_clean_job(ls):
    clean_text = list()
    for narrative in ls:
        if narrative is not None and len(narrative) > 0:
            narrative = narrative.strip()
            clean_text.append(narrative)
    return clean_text

df_raw["clean_narrative"] = run_clean_job(df_raw["reviewText"])

df_clean = pd.DataFrame({
    'clean_narrative': df_raw["clean_narrative"],
    'sentiment': df_raw["sentiment"]
})

n_splits = 2
stratified_kfold = StratifiedKFold(n_splits=n_splits, random_state=42,
    ↪shuffle=True)

train_indices_list = []
test_indices_list = []

for train_index, test_index in stratified_kfold.
    ↪split(df_clean['clean_narrative'], df_clean['sentiment']):
    train_indices_list.append(train_index)
    test_indices_list.append(test_index)

split_index = 0
train_indices = train_indices_list[split_index]
test_indices = test_indices_list[split_index]

X_train = df_clean['clean_narrative'].iloc[train_indices]
y_train = df_clean['sentiment'].iloc[train_indices]
X_test = df_clean['clean_narrative'].iloc[test_indices]
y_test = df_clean['sentiment'].iloc[test_indices]

```

```
[97]: from sklearn.model_selection import train_test_split
```

```

X_train, X_test, y_train, y_test = train_test_split(
    df_clean['clean_narrative'],
    df_clean['sentiment'],
    test_size=0.5,
    random_state=42,

```



```
stratify=df_clean['sentiment']
)
```

```
[98]: vectorizer = CountVectorizer()
X_train_vectorized = vectorizer.fit_transform(X_train)
X_test_vectorized = vectorizer.transform(X_test)
```

```
[99]: naive_bayes = MultinomialNB()
naive_bayes.fit(X_train_vectorized, y_train)
```

```
[99]: MultinomialNB()
```

```
[100]: predictions = naive_bayes.predict(X_test_vectorized)
```

```
[101]: print("Accuracy:", accuracy_score(y_test, predictions))
print("\nClassification Report:\n", classification_report(y_test, predictions))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, predictions))
```

Accuracy: 1.0

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 1       |
| 1            | 1.00      | 1.00   | 1.00     | 1       |
| accuracy     |           |        | 1.00     | 2       |
| macro avg    | 1.00      | 1.00   | 1.00     | 2       |
| weighted avg | 1.00      | 1.00   | 1.00     | 2       |

Confusion Matrix:

```
[[1 0]
 [0 1]]
```

```
[102]: print("Unique values in y_test:", y_test.unique())
```

Unique values in y\_test: [1 0]

```
[104]: print("y shape:", df_clean['sentiment'].shape)
```

y shape: (4,)

```
[106]: print("X shape:", X.shape)
print("y shape:", df_clean['sentiment'].shape)
```

X shape: (4915, 1000)

y shape: (4,)

```
[107]: X_dense = X.toarray()
```

```
[124]: !pip install --upgrade setuptools wheel
```

```
Requirement already satisfied: setuptools in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (69.1.1)
Requirement already satisfied: wheel in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (0.38.4)
Collecting wheel
  Using cached wheel-0.42.0-py3-none-any.whl.metadata (2.2 kB)
Using cached wheel-0.42.0-py3-none-any.whl (65 kB)
Installing collected packages: wheel
  Attempting uninstall: wheel
    Found existing installation: wheel 0.38.4
    Uninstalling wheel-0.38.4:
      Successfully uninstalled wheel-0.38.4
Successfully installed wheel-0.42.0
```

```
[ ]: !sudo apt-get install build-essential
```

Password:

```
[123]: !pip install imbalanced-learn==0.12.0
!pip install scikit-learn==1.0.2
```

```
Requirement already satisfied: imbalanced-learn==0.12.0 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (0.12.0)
Requirement already satisfied: numpy>=1.17.3 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from imbalanced-
learn==0.12.0) (1.24.3)
Requirement already satisfied: scipy>=1.5.0 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from imbalanced-
learn==0.12.0) (1.11.1)
Requirement already satisfied: scikit-learn>=1.0.2 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from imbalanced-
learn==0.12.0) (1.4.1.post1)
Requirement already satisfied: joblib>=1.1.1 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from imbalanced-
learn==0.12.0) (1.2.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/Users/monicafam/anaconda3/lib/python3.11/site-packages (from imbalanced-
learn==0.12.0) (2.2.0)
Collecting scikit-learn==1.0.2
  Downloading scikit-learn-1.0.2.tar.gz (6.7 MB)
    6.7/6.7 MB
7.4 MB/s eta 0:00:0000:0100:01m
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... error
```

**error:** subprocess-exited-with-error

× Preparing metadata

(pyproject.toml) did not run successfully.

exit code: 1

> [1494 lines of output]

Partial import of sklearn during the build process.

setup.py:128: DeprecationWarning:

    `numpy.distutils` is deprecated since NumPy 1.23.0, as a result  
    of the deprecation of `distutils` itself. It will be removed

for

    Python >= 3.12. For older Python versions it will remain  
present.

    It is recommended to use `setuptools < 60.0` for those Python  
versions.

    For more details, see:

[https://numpy.org/devdocs/reference/distutils\\_status\\_migration.html](https://numpy.org/devdocs/reference/distutils_status_migration.html)

```
from numpy.distutils.command.build_ext import build_ext # noqa
INFO: C compiler: clang -DNDEBUG -fwrapv -O2 -Wall -fPIC -O2
-isystem /Users/monicafam/anaconda3/include -fPIC -O2 -isystem
/Users/monicafam/anaconda3/include
```

```
INFO: compile options: '-c'
INFO: clang: test_program.c
INFO: clang objects/test_program.o -o test_program
INFO: C compiler: clang -DNDEBUG -fwrapv -O2 -Wall -fPIC -O2
-isystem /Users/monicafam/anaconda3/include -fPIC -O2 -isystem
/Users/monicafam/anaconda3/include
```

```
INFO: compile options: '-c'
extra options: '-fopenmp'
INFO: clang: test_program.c
clang: error: unsupported option '-fopenmp'
/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-
install-z51vvxl8/scikit-learn_25af3e2111f84aedb79a9332bdaff2c8/sklearn/_build_ut
ils/openmp_helpers.py:127: UserWarning:
```

```
*****
* WARNING *
*****
```

It seems that scikit-learn cannot be built with OpenMP.

- Make sure you have followed the installation instructions:

[https://scikit-learn.org/dev/developers/advanced\\_installation.html](https://scikit-learn.org/dev/developers/advanced_installation.html)

- If your compiler supports OpenMP but you still see this message, please submit a bug report at:

<https://github.com/scikit-learn/scikit-learn/issues>

- The build will continue with OpenMP-based parallelism disabled. Note however that some estimators will run in sequential mode instead of leveraging thread-based parallelism.

\*\*\*

```
warnings.warn(message)
Compiling sklearn/__check_build/_check_build.pyx because it
changed.
Compiling sklearn/preprocessing/_csr_polynomial_expansion.pyx
because it changed.
Compiling sklearn/cluster/_dbscan_inner.pyx because it changed.
Compiling sklearn/cluster/_hierarchical_fast.pyx because it
changed.
Compiling sklearn/cluster/_k_means_common.pyx because it changed.
Compiling sklearn/cluster/_k_means_lloyd.pyx because it changed.
Compiling sklearn/cluster/_k_means_elkan.pyx because it changed.
Compiling sklearn/cluster/_k_means_minibatch.pyx because it
changed.
Compiling sklearn/datasets/_svmlight_format_fast.pyx because it
changed.
Compiling sklearn/decomposition/_online_lda_fast.pyx because it
changed.
Compiling sklearn/decomposition/_cdnmf_fast.pyx because it
changed.
Compiling sklearn/ensemble/_gradient_boosting.pyx because it
changed.
Compiling
sklearn/ensemble/_hist_gradient_boosting/_gradient_boosting.pyx because it
changed.
Compiling sklearn/ensemble/_hist_gradient_boosting/histogram.pyx
because it changed.
Compiling sklearn/ensemble/_hist_gradient_boosting/splitting.pyx
because it changed.
Compiling sklearn/ensemble/_hist_gradient_boosting/binning.pyx
because it changed.
Compiling sklearn/ensemble/_hist_gradient_boosting/predictor.pyx
```

because it changed.

    Compiling sklearn/ensemble/\_hist\_gradient\_boosting/\_loss.pyx  
because it changed.

    Compiling sklearn/ensemble/\_hist\_gradient\_boosting/\_bitset.pyx  
because it changed.

    Compiling sklearn/ensemble/\_hist\_gradient\_boosting/common.pyx  
because it changed.

    Compiling sklearn/ensemble/\_hist\_gradient\_boosting/utils.pyx  
because it changed.

    Compiling sklearn/feature\_extraction/\_hashing\_fast.pyx because it  
changed.

    Compiling sklearn/manifold/\_utils.pyx because it changed.

    Compiling sklearn/manifold/\_barnes\_hut\_tsne.pyx because it  
changed.

    Compiling sklearn/metrics/cluster/\_expected\_mutual\_info\_fast.pyx  
because it changed.

    Compiling sklearn/metrics/\_pairwise\_fast.pyx because it changed.

    Compiling sklearn/metrics/\_dist\_metrics.pyx because it changed.

    Compiling sklearn/neighbors/\_ball\_tree.pyx because it changed.

    Compiling sklearn/neighbors/\_kd\_tree.pyx because it changed.

    Compiling sklearn/neighbors/\_partition\_nodes.pyx because it  
changed.

    Compiling sklearn/neighbors/\_quad\_tree.pyx because it changed.

    Compiling sklearn/tree/\_tree.pyx because it changed.

    Compiling sklearn/tree/\_splitter.pyx because it changed.

    Compiling sklearn/tree/\_criterion.pyx because it changed.

    Compiling sklearn/tree/\_utils.pyx because it changed.

    Compiling sklearn/utils/sparsefuncs\_fast.pyx because it changed.

    Compiling sklearn/utils/\_cython\_blas.pyx because it changed.

    Compiling sklearn/utils/arrayfuncs.pyx because it changed.

    Compiling sklearn/utils/murmurhash.pyx because it changed.

    Compiling sklearn/utils/\_fast\_dict.pyx because it changed.

    Compiling sklearn/utils/\_openmp\_helpers.pyx because it changed.

    Compiling sklearn/utils/\_seq\_dataset.pyx because it changed.

    Compiling sklearn/utils/\_weight\_vector.pyx because it changed.

    Compiling sklearn/utils/\_random.pyx because it changed.

    Compiling sklearn/utils/\_logistic\_sigmoid.pyx because it changed.

    Compiling sklearn/utils/\_readonly\_array\_wrapper.pyx because it  
changed.

    Compiling sklearn/utils/\_typedefs.pyx because it changed.

    Compiling sklearn/svm/\_newrand.pyx because it changed.

    Compiling sklearn/svm/\_libsvm.pyx because it changed.

    Compiling sklearn/svm/\_liblinear.pyx because it changed.

    Compiling sklearn/svm/\_libsvm\_sparse.pyx because it changed.

    Compiling sklearn/linear\_model/\_cd\_fast.pyx because it changed.

    Compiling sklearn/linear\_model/\_sgd\_fast.pyx because it changed.

    Compiling sklearn/linear\_model/\_sag\_fast.pyx because it changed.

    Compiling sklearn/\_isotonic.pyx because it changed.

warning: sklearn/metrics/\_dist\_metrics.pxd:12:64: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:22:65: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:31:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:35:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:54:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:57:52: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:64:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:66:67: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:27:5:  
Exception check on '\_euclidean\_dense\_dense' will always require the GIL to be acquired. Declare '\_euclidean\_dense\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:59:5:  
Exception check on '\_euclidean\_sparse\_dense' will always require the GIL to be acquired. Declare '\_euclidean\_sparse\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:116:40:  
Exception check after calling '\_\_pyx\_fuse\_0\_euclidean\_dense\_dense' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_0\_euclidean\_dense\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:116:40:  
Exception check after calling '\_\_pyx\_fuse\_1\_euclidean\_dense\_dense' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1\_euclidean\_dense\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:150:41:  
Exception check after calling '\_\_pyx\_fuse\_0\_euclidean\_sparse\_dense' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_0\_euclidean\_sparse\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_common.pyx:150:41:

Exception check after calling `'__pyx_fuse_1_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:334:5:

Exception check on `'_update_chunk_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'_update_chunk_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'_update_chunk_dense'` to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:569:5:

Exception check on `'_update_chunk_sparse'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'_update_chunk_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'_update_chunk_sparse'` to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:86:41:

Exception check after calling `'__pyx_fuse_0_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:91:45:

Exception check after calling `'__pyx_fuse_0_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:86:41:

Exception check after calling `'__pyx_fuse_1_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:91:45:

Exception check after calling `'__pyx_fuse_1_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:162:42:

Exception check after calling `'__pyx_fuse_0_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:170:46:

Exception check after calling `'__pyx_fuse_0_euclidean_sparse_dense'` will always

require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:162:42:

Exception check after calling `'__pyx_fuse_1_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:170:46:

Exception check after calling `'__pyx_fuse_1_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:292:31:

Exception check after calling `'__pyx_fuse_0_update_chunk_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_update_chunk_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_update_chunk_dense'` to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:292:31:

Exception check after calling `'__pyx_fuse_1_update_chunk_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_update_chunk_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_update_chunk_dense'` to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:380:60:

Exception check after calling `'__pyx_fuse_0_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:391:57:

Exception check after calling `'__pyx_fuse_0_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:380:60:

Exception check after calling `'__pyx_fuse_1_euclidean_dense_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/cluster/\_k\_means\_elkan.pyx:391:57:

Exception check after calling `'__pyx_fuse_1_euclidean_dense_dense'` will always



require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_dense_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:523:32:`

Exception check after calling `'__pyx_fuse_0_update_chunk_sparse'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_update_chunk_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_update_chunk_sparse'` to allow an error code to be returned.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:523:32:`

Exception check after calling `'__pyx_fuse_1_update_chunk_sparse'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_update_chunk_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_update_chunk_sparse'` to allow an error code to be returned.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:619:61:`

Exception check after calling `'__pyx_fuse_0_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:631:58:`

Exception check after calling `'__pyx_fuse_0_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:619:61:`

Exception check after calling `'__pyx_fuse_1_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/cluster/_k_means_elkan.pyx:631:58:`

Exception check after calling `'__pyx_fuse_1_euclidean_sparse_dense'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_euclidean_sparse_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/cluster/_k_means_lloyd.pyx:162:5:`

Exception check on `'_update_chunk_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'_update_chunk_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'_update_chunk_dense'` to allow

an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:356:5:

Exception check on '\_update\_chunk\_sparse' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_update\_chunk\_sparse' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_update\_chunk\_sparse' to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:129:31:

Exception check after calling '\_\_pyx\_fuse\_0\_update\_chunk\_dense' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_update\_chunk\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_update\_chunk\_dense' to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:129:31:

Exception check after calling '\_\_pyx\_fuse\_1\_update\_chunk\_dense' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1\_update\_chunk\_dense' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_1\_update\_chunk\_dense' to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:196:9:

Exception check after calling '\_\_pyx\_fuse\_0\_gemm' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_gemm' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_gemm' to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:196:9:

Exception check after calling '\_\_pyx\_fuse\_1\_gemm' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1\_gemm' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_1\_gemm' to allow an error code to be returned.

performance hint: sklearn/cluster/\_k\_means\_lloyd.pyx:322:32:

Exception check after calling '\_\_pyx\_fuse\_0\_update\_chunk\_sparse' will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_update_chunk_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_update_chunk_sparse'` to allow an error code to be returned.  
performance hint: `sklearn/cluster/_k_means_lloyd.pyx:322:32:`  
Exception check after calling `'__pyx_fuse_1_update_chunk_sparse'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_update_chunk_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_update_chunk_sparse'` to allow an error code to be returned.  
performance hint: `sklearn/cluster/_k_means_minibatch.pyx:67:5:`  
Exception check on `'update_center_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'update_center_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'update_center_dense'` to allow an error code to be returned.  
performance hint: `sklearn/cluster/_k_means_minibatch.pyx:175:5:`  
Exception check on `'update_center_sparse'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'update_center_sparse'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'update_center_sparse'` to allow an error code to be returned.  
performance hint: `sklearn/cluster/_k_means_minibatch.pyx:60:31:`  
Exception check after calling `'__pyx_fuse_0update_center_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0update_center_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0update_center_dense'` to allow an error code to be returned.  
performance hint: `sklearn/cluster/_k_means_minibatch.pyx:60:31:`  
Exception check after calling `'__pyx_fuse_1update_center_dense'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1update_center_dense'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on

'\_\_pyx\_fuse\_lupdate\_center\_dense' to allow an error code to be returned.  
performance hint: sklearn/cluster/\_k\_means\_minibatch.pyx:168:32:  
Exception check after calling '\_\_pyx\_fuse\_0update\_center\_sparse' will always  
require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0update\_center\_sparse' as 'noexcept'  
if you control the definition and you're sure you don't want the function to  
raise exceptions.

2. Use an 'int' return type on  
'\_\_pyx\_fuse\_0update\_center\_sparse' to allow an error code to be returned.  
performance hint: sklearn/cluster/\_k\_means\_minibatch.pyx:168:32:  
Exception check after calling '\_\_pyx\_fuse\_lupdate\_center\_sparse' will always  
require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_lupdate\_center\_sparse' as 'noexcept'  
if you control the definition and you're sure you don't want the function to  
raise exceptions.

2. Use an 'int' return type on  
'\_\_pyx\_fuse\_lupdate\_center\_sparse' to allow an error code to be returned.  
warning: sklearn/tree/\_tree.pxd:61:73: The keyword 'nogil' should  
appear at the end of the function signature line. Placing it before 'except' or  
'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:62:59: The keyword 'nogil' should  
appear at the end of the function signature line. Placing it before 'except' or  
'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:63:63: The keyword 'nogil' should  
appear at the end of the function signature line. Placing it before 'except' or  
'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:84:72: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:89:68: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:57:45: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:58:40: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:59:48: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:60:57: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before  
'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:49:75: The keyword 'nogil'  
should appear at the end of the function signature line. Placing it before

'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:87:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:119:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:137:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:139:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:160:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:161:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint:  
sklearn/ensemble/\_hist\_gradient\_boosting/\_bitset.pyx:15:5: Exception check on 'init\_bitset' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'init\_bitset' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'init\_bitset' to allow an error code to be returned.

performance hint:  
sklearn/ensemble/\_hist\_gradient\_boosting/\_bitset.pyx:23:5: Exception check on 'set\_bitset' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'set\_bitset' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'set\_bitset' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_loss.pyx:193:5: Exception check on '\_compute\_softmax' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_compute\_softmax' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_compute\_softmax' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_loss.pyx:173:28: Exception check after calling '\_compute\_softmax' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_compute\_softmax' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_compute\_softmax' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_loss.pyx:184:28: Exception check after calling '\_compute\_softmax' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_compute\_softmax' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_compute\_softmax' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_predictor.pyx:69:38: Exception check after calling 'in\_bitset\_2d\_memoryview' will always require the GIL to be acquired. Declare 'in\_bitset\_2d\_memoryview' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_predictor.pyx:74:40: Exception check after calling 'in\_bitset\_2d\_memoryview' will always require the GIL to be acquired. Declare 'in\_bitset\_2d\_memoryview' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/\_predictor.pyx:135:38: Exception check after calling 'in\_bitset\_2d\_memoryview' will always require the GIL to be acquired. Declare 'in\_bitset\_2d\_memoryview' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:257:6: Exception check on '\_build\_histogram\_naive' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_build\_histogram\_naive' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_build\_histogram\_naive' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:281:6: Exception check on '\_subtract\_histograms' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_subtract\_histograms' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_subtract\_histograms' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:305:6: Exception check on '\_build\_histogram' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_build\_histogram' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_build\_histogram' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:352:6: Exception check on '\_build\_histogram\_no\_hessian' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_build\_histogram\_no\_hessian' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_build\_histogram\_no\_hessian' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:396:6: Exception check on '\_build\_histogram\_root' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_build\_histogram\_root' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_build\_histogram\_root' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:449:6: Exception check on '\_build\_histogram\_root\_no\_hessian' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_build\_histogram\_root\_no\_hessian' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_build\_histogram\_root\_no\_hessian' to allow an error code to be returned.

performance hint:

sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx:161:60: Exception check

after calling `'_compute_histogram_brute_single_feature'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'_compute_histogram_brute_single_feature'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.
  2. Use an `'int'` return type on `'_compute_histogram_brute_single_feature'` to allow an error code to be returned.
- performance hint:

`sklearn/ensemble/_hist_gradient_boosting/histogram.pyx:193:48: Exception check after calling '_build_histogram_root_no_hessian' will always require the GIL to be acquired.`

Possible solutions:

1. Declare `'_build_histogram_root_no_hessian'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.
  2. Use an `'int'` return type on `'_build_histogram_root_no_hessian'` to allow an error code to be returned.
- performance hint:

`sklearn/ensemble/_hist_gradient_boosting/histogram.pyx:197:37: Exception check after calling '_build_histogram_root' will always require the GIL to be acquired.`

Possible solutions:

1. Declare `'_build_histogram_root'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.
  2. Use an `'int'` return type on `'_build_histogram_root'` to allow an error code to be returned.
- performance hint:

`sklearn/ensemble/_hist_gradient_boosting/histogram.pyx:202:43: Exception check after calling '_build_histogram_no_hessian' will always require the GIL to be acquired.`

Possible solutions:

1. Declare `'_build_histogram_no_hessian'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.
  2. Use an `'int'` return type on `'_build_histogram_no_hessian'` to allow an error code to be returned.
- performance hint:

`sklearn/ensemble/_hist_gradient_boosting/histogram.pyx:206:32: Exception check after calling '_build_histogram' will always require the GIL to be acquired.`

Possible solutions:

1. Declare `'_build_histogram'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.
  2. Use an `'int'` return type on `'_build_histogram'` to allow an error code to be returned.
- performance hint:

`sklearn/ensemble/_hist_gradient_boosting/histogram.pyx:249:32: Exception check`



after calling '\_subtract\_histograms' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_subtract\_histograms' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_subtract\_histograms' to allow an error code to be returned.

Error compiling Cython file:

-----

...

```
    if n_used_bins <= 1:
        free(cat_infos)
        return
```

```
    qsort(cat_infos, n_used_bins, sizeof(categorical_info),
          compare_cat_infos)
    ^
```

-----

sklearn/ensemble/\_hist\_gradient\_boosting/splitting.pyx:920:14:  
Cannot assign type 'int (const void \*, const void \*) except? -1 nogil' to 'int (\*) (const void \*, const void \*) noexcept nogil'. Exception values are incompatible. Suggest adding 'noexcept' to type 'int (const void \*, const void \*) except? -1 nogil'.

Traceback (most recent call last):

File

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-  
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",  
line 1345, in cythonize_one_helper  
    return cythonize_one(*m)  
    ~~~~~~
```

File

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-  
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",  
line 1321, in cythonize_one
```

```
    raise CompileError(None, pyx_file)
```

Cython.Compiler.Errors.CompileError:

sklearn/ensemble/\_hist\_gradient\_boosting/splitting.pyx

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:46:5:

Exception check on 'fmax' will always require the GIL to be acquired. Declare 'fmax' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:52:5:

Exception check on 'fsign' will always require the GIL to be acquired. Declare 'fsign' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:61:5:

Exception check on 'abs\_max' will always require the GIL to be acquired. Declare 'abs\_max' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:73:5:
```

Exception check on 'max' will always require the GIL to be acquired. Declare 'max' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:85:5:
```

Exception check on 'diff\_abs\_max' will always require the GIL to be acquired. Declare 'diff\_abs\_max' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:154:13:
```

Exception check after calling '\_\_pyx\_fuse\_0\_copy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_copy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_copy' to allow an error code to be returned.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:155:13:
```

Exception check after calling '\_\_pyx\_fuse\_0\_gemv' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_gemv' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_gemv' to allow an error code to be returned.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:159:19:
```

Exception check after calling '\_\_pyx\_fuse\_0\_dot' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_0\_dot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:177:25:
```

Exception check after calling '\_\_pyx\_fuse\_0\_axpy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_axpy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_axpy' to allow an error code to be returned.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:180:26:
```

Exception check after calling '\_\_pyx\_fuse\_0\_dot' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_0\_dot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:185:34:
```

Exception check after calling '\_\_pyx\_fuse\_0\_fsign' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_0\_fsign' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

```
performance hint: sklearn/linear_model/_cd_fast.pyx:185:46:
```

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:190:25:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:194:30:

Exception check after calling `'__pyx_fuse_0fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:196:28:

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:206:21:

Exception check after calling `'__pyx_fuse_0_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:207:21:

Exception check after calling `'__pyx_fuse_0_gemv'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_gemv'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_gemv'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:213:39:

Exception check after calling `'__pyx_fuse_0max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:215:43:

Exception check after calling `'__pyx_fuse_0abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:218:30:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:221:30:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:231:31:

Exception check after calling `'__pyx_fuse_0_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:235:38:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:154:13:

Exception check after calling `'__pyx_fuse_1_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:155:13:

Exception check after calling `'__pyx_fuse_1_gemv'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_gemv'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_gemv'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:159:19:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:177:25:

Exception check after calling `'__pyx_fuse_1_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:180:26:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:185:34:

Exception check after calling `'__pyx_fuse_1fsign'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fsign'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:185:46:

Exception check after calling '`__pyx_fuse_1fmax`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1fmax`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:190:25:

Exception check after calling '`__pyx_fuse_1axpy`' will always require the GIL to be acquired.

Possible solutions:

1. Declare '`__pyx_fuse_1axpy`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an '`int`' return type on '`__pyx_fuse_1axpy`' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:194:30:

Exception check after calling '`__pyx_fuse_1fmax`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1fmax`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:196:28:

Exception check after calling '`__pyx_fuse_1fmax`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1fmax`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:206:21:

Exception check after calling '`__pyx_fuse_1copy`' will always require the GIL to be acquired.

Possible solutions:

1. Declare '`__pyx_fuse_1copy`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an '`int`' return type on '`__pyx_fuse_1copy`' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:207:21:

Exception check after calling '`__pyx_fuse_1gemv`' will always require the GIL to be acquired.

Possible solutions:

1. Declare '`__pyx_fuse_1gemv`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an '`int`' return type on '`__pyx_fuse_1gemv`' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:213:39:

Exception check after calling '`__pyx_fuse_1max`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1max`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:215:43:

Exception check after calling '`__pyx_fuse_1abs_max`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1abs_max`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:218:30:

Exception check after calling '`__pyx_fuse_1dot`' will always require the GIL to be acquired. Declare '`__pyx_fuse_1dot`' as '`noexcept`' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:221:30:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:231:31:`

Exception check after calling `'__pyx_fuse_1_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:235:38:`

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:353:19:`

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:396:33:`

Exception check after calling `'__pyx_fuse_0fsign'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0fsign'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:396:45:`

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:437:39:`

Exception check after calling `'__pyx_fuse_0max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:439:43:`

Exception check after calling `'__pyx_fuse_0abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:442:30:`

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:445:30:`

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:454:31:`

Exception check after calling `'__pyx_fuse_0_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:456:54:`

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:353:19:`

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:396:33:

Exception check after calling `'__pyx_fuse_1fsign'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fsign'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:396:45:

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:437:39:

Exception check after calling `'__pyx_fuse_1max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:439:43:

Exception check after calling `'__pyx_fuse_1abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:442:30:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:445:30:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:454:31:

Exception check after calling `'__pyx_fuse_1_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:456:54:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:555:25:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:563:33:

Exception check after calling `'__pyx_fuse_0fsign'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0fsign'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:563:45:

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:568:25:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:585:30:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:590:39:

Exception check after calling `'__pyx_fuse_0max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:592:43:

Exception check after calling `'__pyx_fuse_0abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:601:30:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:612:37:

Exception check after calling `'__pyx_fuse_0_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:555:25:

Exception check after calling `'__pyx_fuse_1_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:563:33:

Exception check after calling `'__pyx_fuse_1fsign'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fsign'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:563:45:

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:568:25:



Exception check after calling `'__pyx_fuse_1_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:585:30:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:590:39:

Exception check after calling `'__pyx_fuse_1max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:592:43:

Exception check after calling `'__pyx_fuse_1abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:601:30:

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:612:37:

Exception check after calling `'__pyx_fuse_1_asum'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_asum'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:694:35:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:697:13:

Exception check after calling `'__pyx_fuse_0_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:701:25:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:705:25:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:720:21:

Exception check after calling `'__pyx_fuse_0_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:732:29:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:742:34:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:746:26:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:749:21:

Exception check after calling `'__pyx_fuse_0_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:750:35:

Exception check after calling `'__pyx_fuse_1fmax'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1fmax'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:750:21:

Exception check after calling `'__pyx_fuse_0_scal'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_scal'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_scal'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:763:29:

Exception check after calling `'__pyx_fuse_0_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_0_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_0_axpy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:767:37:

Exception check after calling `'__pyx_fuse_0diff_abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0diff_abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:772:38:

Exception check after calling `'__pyx_fuse_0abs_max'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0abs_max'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:784:42:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:792:41:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:799:30:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:800:30:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:810:29:

Exception check after calling `'__pyx_fuse_0_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:815:37:

Exception check after calling `'__pyx_fuse_0_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_0_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:694:35:

Exception check after calling `'__pyx_fuse_1_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:697:13:

Exception check after calling `'__pyx_fuse_1_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_copy'` to allow an error code to be returned.

performance hint: `sklearn/linear_model/_cd_fast.pyx:701:25:`

Exception check after calling `'__pyx_fuse_1_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_axpy'` to allow an error code to be returned.

performance hint: `sklearn/linear_model/_cd_fast.pyx:705:25:`

Exception check after calling `'__pyx_fuse_1_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:720:21:`

Exception check after calling `'__pyx_fuse_1_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_copy'` to allow an error code to be returned.

performance hint: `sklearn/linear_model/_cd_fast.pyx:732:29:`

Exception check after calling `'__pyx_fuse_1_axpy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_axpy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_axpy'` to allow an error code to be returned.

performance hint: `sklearn/linear_model/_cd_fast.pyx:742:34:`

Exception check after calling `'__pyx_fuse_1_dot'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_dot'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:746:26:`

Exception check after calling `'__pyx_fuse_1_nrm2'` will always require the GIL to be acquired. Declare `'__pyx_fuse_1_nrm2'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: `sklearn/linear_model/_cd_fast.pyx:749:21:`

Exception check after calling `'__pyx_fuse_1_copy'` will always require the GIL to be acquired.

Possible solutions:

1. Declare `'__pyx_fuse_1_copy'` as `'noexcept'` if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an `'int'` return type on `'__pyx_fuse_1_copy'` to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:750:35:  
Exception check after calling '\_\_pyx\_fuse\_1fmax' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1fmax' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:750:21:  
Exception check after calling '\_\_pyx\_fuse\_1scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_1scal' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:763:29:  
Exception check after calling '\_\_pyx\_fuse\_1axpy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1axpy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_1axpy' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:767:37:  
Exception check after calling '\_\_pyx\_fuse\_1diff\_abs\_max' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1diff\_abs\_max' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:772:38:  
Exception check after calling '\_\_pyx\_fuse\_1abs\_max' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1abs\_max' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:784:42:  
Exception check after calling '\_\_pyx\_fuse\_1dot' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1dot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:792:41:  
Exception check after calling '\_\_pyx\_fuse\_1nrm2' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1nrm2' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:799:30:  
Exception check after calling '\_\_pyx\_fuse\_1nrm2' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1nrm2' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:800:30:  
Exception check after calling '\_\_pyx\_fuse\_1nrm2' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1nrm2' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:810:29:  
Exception check after calling '\_\_pyx\_fuse\_1dot' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1dot' as 'noexcept' if you control the

definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_cd\_fast.pyx:815:37:

Exception check after calling '\_\_pyx\_fuse\_1\_nrm2' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_1\_nrm2' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1198:5:

Exception check on 'predict\_sample64' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'predict\_sample64' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'predict\_sample64' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1245:5:

Exception check on 'predict\_sample32' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'predict\_sample32' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'predict\_sample32' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:463:43:

Exception check after calling 'random' will always require the GIL to be acquired. Declare 'random' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:491:32:

Exception check after calling 'predict\_sample64' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'predict\_sample64' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'predict\_sample64' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:496:35:

Exception check after calling 'dloss' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'dloss' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:499:44:

Exception check after calling 'dloss' will always require the GIL to be acquired. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:794:43:

Exception check after calling 'random' will always require the GIL to be acquired. Declare 'random' as 'noexcept' if you control the definition and

you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:822:32:

Exception check after calling 'predict\_sample32' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'predict\_sample32' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'predict\_sample32' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:827:35:

Exception check after calling 'dloss' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'dloss' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:830:44:

Exception check after calling 'dloss' will always require the GIL to be acquired. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1333:24:

Exception check after calling 'next' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'next' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'next' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1337:28:

Exception check after calling 'predict\_sample64' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'predict\_sample64' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'predict\_sample64' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1341:27:

Exception check after calling 'dloss' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'dloss' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sag\_fast.pyx:1344:39:

Exception check after calling '\_loss' will always require the GIL to be acquired. Declare '\_loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:704:5:  
Exception check on 'l1penalty' will always require the GIL to be acquired.  
Possible solutions:

1. Declare 'l1penalty' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'l1penalty' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:547:31:  
Exception check after calling 'shuffle' will always require the GIL to be acquired.  
Possible solutions:

1. Declare 'shuffle' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'shuffle' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:549:28:  
Exception check after calling 'next' will always require the GIL to be acquired.  
Possible solutions:

1. Declare 'next' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'next' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:557:25:  
Exception check after calling 'dot' will always require the GIL to be acquired.  
Declare 'dot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:564:40:  
Exception check after calling 'loss' will always require the GIL to be acquired.  
Declare 'loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:575:45:  
Exception check after calling 'loss' will always require the GIL to be acquired.  
Declare 'loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:578:38:  
Exception check after calling 'loss' will always require the GIL to be acquired.  
Declare 'loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:580:38:  
Exception check after calling 'dloss' will always require the GIL to be acquired. Declare 'dloss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:602:27:  
Exception check after calling 'scale' will always require the GIL to be acquired.  
Possible solutions:

1. Declare 'scale' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.



2. Use an 'int' return type on 'scale' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:605:25:  
Exception check after calling 'add' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'add' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'add' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:618:33:  
Exception check after calling 'add\_average' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'add\_average' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'add\_average' to allow an error code to be returned.

performance hint: sklearn/linear\_model/\_sgd\_fast.pyx:625:29:  
Exception check after calling 'l1penalty' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'l1penalty' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'l1penalty' to allow an error code to be returned.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/manifold/\_barnes\_hut\_tsne.pyx:220:30:  
Exception check after calling 'summarize' will always require the GIL to be acquired. Declare 'summarize' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

warning: sklearn/metrics/\_dist\_metrics.pxd:12:64: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:22:65: The keyword







'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:562:66: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:570:49: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:632:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1122:58: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1131:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1716:78: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_ball\_tree.pyx:104:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_ball\_tree.pyx:120:82: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_ball\_tree.pyx:131:58: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:12:64: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:22:65: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:31:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:35:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:54:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:57:52: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:64:68: The keyword

'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:66:67: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/neighbors/\_binary\_tree.pxi:506:5:  
Exception check on 'dual\_swap' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'dual\_swap' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'dual\_swap' to allow an error code to be returned.

Error compiling Cython file:

```
-----
...
        # determine number of levels in the tree, and from this
        # the number of nodes in the tree.  This results in leaf
nodes
        # with numbers of points between leaf_size and 2 *
leaf_size
        self.n_levels = int(
            np.log2(fmax(1, (n_samples - 1) / self.leaf_size)) +
1)
        self.n_nodes = (2 ** self.n_levels) - 1
                        ^
-----

sklearn/neighbors/_binary_tree.pxi:982:44: Cannot assign type
'double' to 'ITYPE_t'
Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
           ~~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError:
sklearn/neighbors/_ball_tree.pyx
warning: sklearn/neighbors/_binary_tree.pxi:562:66: The keyword
'nogil' should appear at the end of the function signature line. Placing it
before 'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/neighbors/_binary_tree.pxi:570:49: The keyword
```

'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:632:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1122:58: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1131:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_binary\_tree.pxi:1716:78: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_kd\_tree.pyx:86:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_kd\_tree.pyx:147:82: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:12:64: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:22:65: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:31:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:35:79: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:54:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:57:52: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:64:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/metrics/\_dist\_metrics.pxd:66:67: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/neighbors/\_binary\_tree.pxi:506:5:  
Exception check on 'dual\_swap' will always require the GIL to be acquired.  
Possible solutions:

1. Declare 'dual\_swap' as 'noexcept' if you control the

definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'dual\_swap' to allow an error code to be returned.

```
Error compiling Cython file:
-----
...
        # determine number of levels in the tree, and from this
        # the number of nodes in the tree.  This results in leaf
nodes
        # with numbers of points between leaf_size and 2 *
leaf_size
        self.n_levels = int(
            np.log2(fmax(1, (n_samples - 1) / self.leaf_size)) +
1)
        self.n_nodes = (2 ** self.n_levels) - 1
                        ^
-----

sklearn/neighbors/_binary_tree.pxi:982:44: Cannot assign type
'double' to 'ITYPE_t'
Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
           ~~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError:
sklearn/neighbors/_kd_tree.pyx
warning: sklearn/neighbors/_quad_tree.pxd:72:59: The keyword
'nogil' should appear at the end of the function signature line. Placing it
before 'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/neighbors/_quad_tree.pxd:91:51: The keyword
'nogil' should appear at the end of the function signature line. Placing it
before 'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/neighbors/_quad_tree.pxd:94:59: The keyword
'nogil' should appear at the end of the function signature line. Placing it
before 'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/neighbors/_quad_tree.pxd:95:63: The keyword
'nogil' should appear at the end of the function signature line. Placing it
before 'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/neighbors/_quad_tree.pxd:96:80: The keyword
```



'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pyx:116:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pyx:305:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pyx:464:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pyx:559:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pyx:571:70: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:49:75: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:87:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:119:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:137:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:139:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:160:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:161:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:61:73: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:62:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:63:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:84:72: The keyword 'nogil'

should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:89:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:57:45: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:58:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:59:48: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:60:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

Error compiling Cython file:

-----

...

```
def __cinit__(self, int n_dimensions, int verbose):
    """Constructor."""
    # Parameters of the tree
    self.n_dimensions = n_dimensions
    self.verbose = verbose
    self.n_cells_per_cell = 2 ** self.n_dimensions
    ^
```

-----

sklearn/neighbors/\_quad\_tree.pyx:56:34: Cannot assign type 'double' to 'SIZE\_t'

Traceback (most recent call last):

File

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
    ~~~~~
```

File

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError:
sklearn/neighbors/_quad_tree.pyx
```

Error compiling Cython file:

```

-----
...
    free_problem(problem)
    free_parameter(param)
    raise ValueError(error_msg)

cdef BlasFunctions blas_functions
blas_functions.dot = _dot[double]
    ^
-----

sklearn/svm/_liblinear.pyx:55:29: Cannot assign type 'double
(int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'double (int, double
*, int, double *, int) except * nogil'.

Error compiling Cython file:
-----
...
    free_parameter(param)
    raise ValueError(error_msg)

cdef BlasFunctions blas_functions
blas_functions.dot = _dot[double]
blas_functions.axy = _axy[double]
    ^
-----

sklearn/svm/_liblinear.pyx:56:31: Cannot assign type 'void (int,
double, double *, int, double *, int) except * nogil' to 'axy_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'void (int, double,
double *, int, double *, int) except * nogil'.

Error compiling Cython file:
-----
...
    raise ValueError(error_msg)

cdef BlasFunctions blas_functions
blas_functions.dot = _dot[double]
blas_functions.axy = _axy[double]
blas_functions.scal = _scal[double]
    ^
-----

sklearn/svm/_liblinear.pyx:57:31: Cannot assign type 'void (int,
double, double *, int) except * nogil' to 'scal_func'. Exception values are
incompatible. Suggest adding 'noexcept' to type 'void (int, double, double *,

```

```
int) except * nogil'.
```

```
Error compiling Cython file:
```

```
-----
```

```
...
```

```
    cdef BlasFunctions blas_functions
    blas_functions.dot = _dot[double]
    blas_functions.axy = _axy[double]
    blas_functions.scal = _scal[double]
    blas_functions.nrm2 = _nrm2[double]
                                ^
```

```
-----
```

```
sklearn/svm/_liblinear.pyx:58:31: Cannot assign type 'double (int, double *, int) except * nogil' to 'nrm2_func'. Exception values are incompatible. Suggest adding 'noexcept' to type 'double (int, double *, int) except * nogil'.
```

```
Traceback (most recent call last):
```

```
File
```

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
           ~~~~~~
```

```
File
```

```
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError: sklearn/svm/_liblinear.pyx
```

```
Error compiling Cython file:
```

```
-----
```

```
...
```

```
    if error_msg:
        # for SVR: epsilon is called p in libsvm
        error_repl = error_msg.decode('utf-8').replace("p < 0",
"epsilon < 0")
        raise ValueError(error_repl)
    cdef BlasFunctions blas_functions
    blas_functions.dot = _dot[double]
                                ^
```

```
-----
```

```
sklearn/svm/_libsvm.pyx:191:29: Cannot assign type 'double (int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception values are incompatible. Suggest adding 'noexcept' to type 'double (int, double *, int,
```

```
double *, int) except * nogil'.
```

```
Error compiling Cython file:
```

```
-----  
...  
                                class_weight_label.data,  
class_weight.data)  
    model = set_model(&param, <int> nSV.shape[0], SV.data,  
SV.shape,  
                                support.data, support.shape,  
sv_coef.strides,  
                                sv_coef.data, intercept.data, nSV.data,  
probA.data, probB.data)  
    cdef BlasFunctions blas_functions  
    blas_functions.dot = _dot[double]  
                                ^  
-----
```

sklearn/svm/\_libsvm.pyx:355:29: Cannot assign type 'double (int, double \*, int, double \*, int) except \* nogil' to 'dot\_func'. Exception values are incompatible. Suggest adding 'noexcept' to type 'double (int, double \*, int, double \*, int) except \* nogil'.

```
Error compiling Cython file:
```

```
-----  
...  
                                sv_coef.data, intercept.data, nSV.data,  
                                probA.data, probB.data)  
  
    cdef np.npy_intp n_class = get_nr(model)  
    cdef BlasFunctions blas_functions  
    blas_functions.dot = _dot[double]  
                                ^  
-----
```

sklearn/svm/\_libsvm.pyx:461:29: Cannot assign type 'double (int, double \*, int, double \*, int) except \* nogil' to 'dot\_func'. Exception values are incompatible. Suggest adding 'noexcept' to type 'double (int, double \*, int, double \*, int) except \* nogil'.

```
Error compiling Cython file:
```

```
-----  
...  
    n_class = 1  
else:  
    n_class = get_nr(model)  
    n_class = n_class * (n_class - 1) // 2  
    cdef BlasFunctions blas_functions
```

```

blas_functions.dot = _dot[double]
~
-----

sklearn/svm/_libsvm.pyx:567:29: Cannot assign type 'double (int,
double *, int, double *, int) except * nogil' to 'dot_func'. Exception values
are incompatible. Suggest adding 'noexcept' to type 'double (int, double *, int,
double *, int) except * nogil'.

Error compiling Cython file:
-----
...
    if error_msg:
        raise ValueError(error_msg)

    cdef np.ndarray[np.float64_t, ndim=1, mode='c'] target
    cdef BlasFunctions blas_functions
    blas_functions.dot = _dot[double]
    ~
-----

sklearn/svm/_libsvm.pyx:711:29: Cannot assign type 'double (int,
double *, int, double *, int) except * nogil' to 'dot_func'. Exception values
are incompatible. Suggest adding 'noexcept' to type 'double (int, double *, int,
double *, int) except * nogil'.
Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
    ~~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError: sklearn/svm/_libsvm.pyx

Error compiling Cython file:
-----
...
    if error_msg:
        free_problem(problem)
        free_param(param)
        raise ValueError(error_msg)
    cdef BlasFunctions blas_functions
    blas_functions.dot = _dot[double]

```

```

-----
sklearn/svm/_libsvm_sparse.pyx:153:29: Cannot assign type 'double
(int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'double (int, double
*, int, double *, int) except * nogil'.

```

Error compiling Cython file:

```

-----
...
                                sv_coef.data, intercept.data,
                                nSV.data, probA.data, probB.data)
#TODO: use check_model
dec_values = np.empty(T_indptr.shape[0]-1)
cdef BlasFunctions blas_functions
blas_functions.dot = _dot[double]
                                ^

```

```

-----
sklearn/svm/_libsvm_sparse.pyx:284:29: Cannot assign type 'double
(int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'double (int, double
*, int, double *, int) except * nogil'.

```

Error compiling Cython file:

```

-----
...
#TODO: use check_model
cdef np.npy_intp n_class = get_nr(model)
cdef int rv
dec_values = np.empty((T_indptr.shape[0]-1, n_class),
dtype=np.float64)
cdef BlasFunctions blas_functions
blas_functions.dot = _dot[double]
                                ^

```

```

-----
sklearn/svm/_libsvm_sparse.pyx:343:29: Cannot assign type 'double
(int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'double (int, double
*, int, double *, int) except * nogil'.

```

Error compiling Cython file:

```

-----
...
n_class = get_nr(model)
n_class = n_class * (n_class - 1) // 2

```

```

        dec_values = np.empty((T_indptr.shape[0] - 1, n_class),
dtype=np.float64)

```

```

        cdef BlasFunctions blas_functions
        blas_functions.dot = _dot[double]

```

```

-----

sklearn/svm/_libsvm_sparse.pyx:412:29: Cannot assign type 'double
(int, double *, int, double *, int) except * nogil' to 'dot_func'. Exception
values are incompatible. Suggest adding 'noexcept' to type 'double (int, double
*, int, double *, int) except * nogil'.

```

Traceback (most recent call last):

File

```

"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper

```

```

    return cythonize_one(*m)
    ~~~~~~

```

File

```

"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one

```

```

    raise CompileError(None, pyx_file)

```

Cython.Compiler.Errors.CompileError:

sklearn/svm/\_libsvm\_sparse.pyx

```

warning: sklearn/tree/_criterion.pxd:57:45: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_criterion.pxd:58:40: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_criterion.pxd:59:48: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_criterion.pxd:60:57: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_tree.pxd:61:73: The keyword 'nogil' should
appear at the end of the function signature line. Placing it before 'except' or
'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_tree.pxd:62:59: The keyword 'nogil' should
appear at the end of the function signature line. Placing it before 'except' or
'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_tree.pxd:63:63: The keyword 'nogil' should
appear at the end of the function signature line. Placing it before 'except' or
'noexcept' will be disallowed in a future version of Cython.

```

```

warning: sklearn/tree/_splitter.pxd:84:72: The keyword 'nogil'

```





should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pyx:1137:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:49:75: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:87:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:119:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:137:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:139:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:160:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:161:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/tree/\_criterion.pyx:165:30: Exception check after calling 'children\_impurity' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'children\_impurity' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'children\_impurity' to allow

an error code to be returned.

performance hint: sklearn/tree/\_criterion.pyx:536:44: Exception check after calling 'log' will always require the GIL to be acquired. Declare 'log' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:570:49: Exception check after calling 'log' will always require the GIL to be acquired. Declare 'log' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:575:50: Exception check after calling 'log' will always require the GIL to be acquired. Declare 'log' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1072:89: Exception check after calling 'get\_median' will always require the GIL to be acquired. Declare 'get\_median' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1100:74: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1102:62: Exception check after calling 'pop' will always require the GIL to be acquired. Declare 'pop' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1128:75: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1130:63: Exception check after calling 'pop' will always require the GIL to be acquired. Declare 'pop' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1168:70: Exception check after calling 'remove' will always require the GIL to be acquired. Declare 'remove' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1184:69: Exception check after calling 'remove' will always require the GIL to be acquired. Declare 'remove' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1248:74: Exception check after calling 'get\_median' will always require the GIL to be acquired. Declare 'get\_median' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1260:75: Exception check after calling 'get\_median' will always require the GIL to be acquired. Declare 'get\_median' as 'noexcept' if you control the definition and you're sure

you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1361:32: Exception check after calling 'poisson\_loss' will always require the GIL to be acquired. Declare 'poisson\_loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1396:56: Exception check after calling 'log' will always require the GIL to be acquired. Declare 'log' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1397:58: Exception check after calling 'log' will always require the GIL to be acquired. Declare 'log' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1418:44: Exception check after calling 'poisson\_loss' will always require the GIL to be acquired. Declare 'poisson\_loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_criterion.pyx:1421:45: Exception check after calling 'poisson\_loss' will always require the GIL to be acquired. Declare 'poisson\_loss' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

warning: sklearn/tree/\_splitter.pxd:84:72: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:89:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:57:45: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:58:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:59:48: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:60:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:61:73: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:62:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:63:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:180:72: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:210:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:264:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:578:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:1096:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pyx:1326:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:49:75: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:87:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:119:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:137:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:139:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:160:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:161:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/tree/\_splitter.pyx:42:5: Exception check on '\_init\_split' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_init\_split' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_init\_split' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:456:5: Exception check on 'sort' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'sort' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'sort' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:463:5: Exception check on 'swap' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'swap' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'swap' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:492:5: Exception check on 'introsort' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'introsort' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'introsort' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:525:5: Exception check on 'sift\_down' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'sift\_down' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'sift\_down' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:548:5: Exception check on 'heapsort' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'heapsort' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'heapsort' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:940:5: Exception  
check on 'binary\_search' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'binary\_search' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'binary\_search' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:965:5: Exception  
check on 'extract\_nnz\_index\_to\_samples' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'extract\_nnz\_index\_to\_samples' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'extract\_nnz\_index\_to\_samples' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:1005:5: Exception  
check on 'extract\_nnz\_binary\_search' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'extract\_nnz\_binary\_search' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'extract\_nnz\_binary\_search' to allow an error code to be returned.

performance hint: sklearn/tree/\_splitter.pyx:1077:5: Exception  
check on 'sparse\_swap' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'sparse\_swap' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'sparse\_swap' to allow an error code to be returned.

Error compiling Cython file:

-----

...

```
if not is_samples_sorted[0]:
    n_samples = end - start
    memcpy(sorted_samples + start, samples + start,
           n_samples * sizeof(SIZE_t))
    qsort(sorted_samples + start, n_samples, sizeof(SIZE_t),
          compare_SIZE_t)
    ^
```

-----

sklearn/tree/\_splitter.pyx:1033:14: Cannot assign type 'int  
(const void \*, const void \*) except? -1 nogil' to 'int (\*)(const void \*, const  
void \*) noexcept nogil'. Exception values are incompatible. Suggest adding  
'noexcept' to type 'int (const void \*, const void \*) except? -1 nogil'.

```

Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
    ~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
  Cython.Compiler.Errors.CompileError: sklearn/tree/_splitter.pyx
  warning: sklearn/tree/_tree.pxd:61:73: The keyword 'nogil' should
  appear at the end of the function signature line. Placing it before 'except' or
  'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_tree.pxd:62:59: The keyword 'nogil' should
  appear at the end of the function signature line. Placing it before 'except' or
  'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_tree.pxd:63:63: The keyword 'nogil' should
  appear at the end of the function signature line. Placing it before 'except' or
  'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_splitter.pxd:84:72: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_splitter.pxd:89:68: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_criterion.pxd:57:45: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_criterion.pxd:58:40: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_criterion.pxd:59:48: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_criterion.pxd:60:57: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_tree.pyx:267:71: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_tree.pyx:414:76: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before
  'except' or 'noexcept' will be disallowed in a future version of Cython.
  warning: sklearn/tree/_tree.pyx:668:59: The keyword 'nogil'
  should appear at the end of the function signature line. Placing it before

```



'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pyx:680:70: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pyx:714:73: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:49:75: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:87:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:119:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:137:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:139:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:160:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pxd:161:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

Error compiling Cython file:

-----  
 ...

```

# Initial capacity
cdef int init_capacity

if tree.max_depth <= 10:
    init_capacity = (2 ** (tree.max_depth + 1)) - 1
    ^
-----

sklearn/tree/_tree.pyx:146:56: Cannot assign type 'double' to
'int'
Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
    ~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError: sklearn/tree/_tree.pyx
warning: sklearn/tree/_utils.pxd:49:75: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:87:61: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:119:56: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:137:40: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:139:71: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:160:71: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_utils.pxd:161:40: The keyword 'nogil'
should appear at the end of the function signature line. Placing it before
'except' or 'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_tree.pxd:61:73: The keyword 'nogil' should
appear at the end of the function signature line. Placing it before 'except' or
'noexcept' will be disallowed in a future version of Cython.
warning: sklearn/tree/_tree.pxd:62:59: The keyword 'nogil' should

```

appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_tree.pxd:63:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:84:72: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_splitter.pxd:89:68: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:57:45: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:58:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:59:48: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_criterion.pxd:60:57: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:72:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:91:51: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:94:59: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:95:63: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/neighbors/\_quad\_tree.pxd:96:80: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:25:75: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:110:61: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:226:56: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:314:40: The keyword 'nogil'

should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:331:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:489:40: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

warning: sklearn/tree/\_utils.pyx:503:71: The keyword 'nogil' should appear at the end of the function signature line. Placing it before 'except' or 'noexcept' will be disallowed in a future version of Cython.

performance hint: sklearn/tree/\_utils.pyx:25:5: Exception check on 'safe\_realloc' will always require the GIL to be acquired. Declare 'safe\_realloc' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:123:24: Exception check after calling '\_\_pyx\_fuse\_9safe\_realloc' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_9safe\_realloc' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:201:27: Exception check after calling 'heapify\_up' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'heapify\_up' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'heapify\_up' to allow an error code to be returned.

performance hint: sklearn/tree/\_utils.pyx:221:29: Exception check after calling 'heapify\_down' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'heapify\_down' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'heapify\_down' to allow an error code to be returned.

performance hint: sklearn/tree/\_utils.pyx:239:24: Exception check after calling '\_\_pyx\_fuse\_10safe\_realloc' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_10safe\_realloc' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:255:23: Exception check after calling 'heapify\_up' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'heapify\_up' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'heapify\_up' to allow an error code to be returned.

performance hint: sklearn/tree/\_utils.pyx:277:29: Exception check after calling 'heapify\_down' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'heapify\_down' as 'noexcept' if you control the

definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'heapify\_down' to allow an error code to be returned.

performance hint: sklearn/tree/\_utils.pyx:322:20: Exception check after calling '\_\_pyx\_fuse\_3safe\_realloc' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_3safe\_realloc' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:345:24: Exception check after calling '\_\_pyx\_fuse\_3safe\_realloc' will always require the GIL to be acquired. Declare '\_\_pyx\_fuse\_3safe\_realloc' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:487:32: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:512:20: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:513:45: Exception check after calling 'get\_median' will always require the GIL to be acquired. Declare 'get\_median' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:516:47: Exception check after calling 'update\_median\_parameters\_post\_push' will always require the GIL to be acquired. Declare 'update\_median\_parameters\_post\_push' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:527:20: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:547:69: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:550:68: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:556:44: Exception check after calling 'size' will always require the GIL to be acquired. Declare 'size' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:559:68: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:569:20: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'  
as 'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:570:45: Exception check  
after calling 'get\_median' will always require the GIL to be acquired. Declare  
'get\_median' as 'noexcept' if you control the definition and you're sure you  
don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:572:42: Exception check  
after calling 'remove' will always require the GIL to be acquired. Declare  
'remove' as 'noexcept' if you control the definition and you're sure you don't  
want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:573:49: Exception check  
after calling 'update\_median\_parameters\_post\_remove' will always require the GIL  
to be acquired. Declare 'update\_median\_parameters\_post\_remove' as 'noexcept' if  
you control the definition and you're sure you don't want the function to raise  
exceptions.

performance hint: sklearn/tree/\_utils.pyx:584:20: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'  
as 'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:585:45: Exception check  
after calling 'get\_median' will always require the GIL to be acquired. Declare  
'get\_median' as 'noexcept' if you control the definition and you're sure you  
don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:588:28: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'  
as 'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:591:39: Exception check  
after calling 'pop' will always require the GIL to be acquired. Declare 'pop' as  
'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:592:49: Exception check  
after calling 'update\_median\_parameters\_post\_remove' will always require the GIL  
to be acquired. Declare 'update\_median\_parameters\_post\_remove' as 'noexcept' if  
you control the definition and you're sure you don't want the function to raise  
exceptions.

performance hint: sklearn/tree/\_utils.pyx:603:28: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'  
as 'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:610:28: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'  
as 'noexcept' if you control the definition and you're sure you don't want the  
function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:631:44: Exception check  
after calling 'size' will always require the GIL to be acquired. Declare 'size'

as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:634:68: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:641:69: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:644:68: Exception check after calling 'get\_weight\_from\_index' will always require the GIL to be acquired. Declare 'get\_weight\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:652:53: Exception check after calling 'get\_value\_from\_index' will always require the GIL to be acquired. Declare 'get\_value\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:653:53: Exception check after calling 'get\_value\_from\_index' will always require the GIL to be acquired. Declare 'get\_value\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/tree/\_utils.pyx:656:52: Exception check after calling 'get\_value\_from\_index' will always require the GIL to be acquired. Declare 'get\_value\_from\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_cython\_blas.pyx:20:5: Exception check on '\_dot' will always require the GIL to be acquired. Declare '\_dot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_cython\_blas.pyx:33:5: Exception check on '\_asum' will always require the GIL to be acquired. Declare '\_asum' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_cython\_blas.pyx:45:5: Exception check on '\_axpy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_axpy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_axpy' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:58:5: Exception check on '\_nrm2' will always require the GIL to be acquired. Declare '\_nrm2' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_cython\_blas.pyx:70:5: Exception check on '\_copy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_copy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_copy' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:82:5: Exception check on '\_scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_scal' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:94:5: Exception check on '\_rotg' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_rotg' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_rotg' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:107:5: Exception check on '\_rot' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_rot' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_rot' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:124:5: Exception check on '\_gemv' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_gemv' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_gemv' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:153:5: Exception check on '\_ger' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_ger' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_ger' to allow an error code to be returned.

performance hint: sklearn/utils/\_cython\_blas.pyx:183:5: Exception check on '\_gemm' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_gemm' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_gemm' to allow an error code to be returned.

Error compiling Cython file:



```

-----
...
    dec(end)
    # Construct our arguments
    cdef pair[ITYPE_t, DTYPE_t] args
    args.first = key
    args.second = value
    self.my_map.insert(end, args)
    ^
-----

sklearn/utils/_fast_dict.pyx:136:27: Cannot assign type
'iterator' to 'const_iterator'
Traceback (most recent call last):
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
    ~~~~~~

  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
Cython.Compiler.Errors.CompileError: sklearn/utils/_fast_dict.pyx
warning: sklearn/utils/_openmp_helpers.pyx:1:0: The 'IF'
statement is deprecated and will be removed in a future Cython version. Consider
using runtime conditions or C macros instead. See
https://github.com/cython/cython/issues/4310
warning: sklearn/utils/_openmp_helpers.pyx:44:4: The 'IF'
statement is deprecated and will be removed in a future Cython version. Consider
using runtime conditions or C macros instead. See
https://github.com/cython/cython/issues/4310
performance hint: sklearn/utils/_seq_dataset.pyx:77:53: Exception
check after calling '_get_next_index' will always require the GIL to be
acquired. Declare '_get_next_index' as 'noexcept' if you control the definition
and you're sure you don't want the function to raise exceptions.
performance hint: sklearn/utils/_seq_dataset.pyx:78:20: Exception
check after calling '_sample' will always require the GIL to be acquired.
Possible solutions:
    1. Declare '_sample' as 'noexcept' if you control the
definition and you're sure you don't want the function to raise exceptions.
    2. Use an 'int' return type on '_sample' to allow an error
code to be returned.
performance hint: sklearn/utils/_seq_dataset.pyx:114:55:
Exception check after calling '_get_random_index' will always require the GIL to
be acquired. Declare '_get_random_index' as 'noexcept' if you control the

```

definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_seq\_dataset.pyx:115:20:

Exception check after calling '\_sample' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_sample' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_sample' to allow an error code to be returned.

performance hint: sklearn/utils/\_seq\_dataset.pyx:402:53:

Exception check after calling '\_get\_next\_index' will always require the GIL to be acquired. Declare '\_get\_next\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_seq\_dataset.pyx:403:20:

Exception check after calling '\_sample' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_sample' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_sample' to allow an error code to be returned.

performance hint: sklearn/utils/\_seq\_dataset.pyx:439:55:

Exception check after calling '\_get\_random\_index' will always require the GIL to be acquired. Declare '\_get\_random\_index' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

performance hint: sklearn/utils/\_seq\_dataset.pyx:440:20:

Exception check after calling '\_sample' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_sample' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_sample' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:178:29:

Exception check after calling 'reset\_wscales' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'reset\_wscales' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on 'reset\_wscales' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:183:17:

Exception check after calling '\_\_pyx\_fuse\_1\_axpy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1\_axpy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.

2. Use an 'int' return type on '\_\_pyx\_fuse\_1\_axpy' to allow

an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:185:17:

Exception check after calling '\_\_pyx\_fuse\_1\_scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1\_scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_1\_scal' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:189:13:

Exception check after calling '\_\_pyx\_fuse\_1\_scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_1\_scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_1\_scal' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:351:29:

Exception check after calling 'reset\_wscales' will always require the GIL to be acquired.

Possible solutions:

1. Declare 'reset\_wscales' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on 'reset\_wscales' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:356:17:

Exception check after calling '\_\_pyx\_fuse\_0\_axpy' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_axpy' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_axpy' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:358:17:

Exception check after calling '\_\_pyx\_fuse\_0\_scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_scal' to allow an error code to be returned.

performance hint: sklearn/utils/\_weight\_vector.pyx:362:13:

Exception check after calling '\_\_pyx\_fuse\_0\_scal' will always require the GIL to be acquired.

Possible solutions:

1. Declare '\_\_pyx\_fuse\_0\_scal' as 'noexcept' if you control the definition and you're sure you don't want the function to raise exceptions.
2. Use an 'int' return type on '\_\_pyx\_fuse\_0\_scal' to allow

an error code to be returned.

```
multiprocessing.pool.RemoteTraceback:
"""

Traceback (most recent call last):
  File
"/Users/monicafam/anaconda3/lib/python3.11/multiprocessing/pool.py", line 125,
in worker
    result = (True, func(*args, **kwargs))
              ~~~~~
  File
"/Users/monicafam/anaconda3/lib/python3.11/multiprocessing/pool.py", line 48, in
mapstar
    return list(map(*args))
              ~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1345, in cythonize_one_helper
    return cythonize_one(*m)
              ~~~~~
  File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1321, in cythonize_one
    raise CompileError(None, pyx_file)
  Cython.Compiler.Errors.CompileError:
sklearn/ensemble/_hist_gradient_boosting/splitting.pyx
"""
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "/Users/monicafam/anaconda3/lib/python3.11/site-
packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 353, in
<module>
    [ 1/55] Cythonizing sklearn/_check_build/_check_build.pyx
    [ 2/55] Cythonizing sklearn/_isotonic.pyx
    [ 3/55] Cythonizing sklearn/cluster/_dbscan_inner.pyx
    [ 4/55] Cythonizing sklearn/cluster/_hierarchical_fast.pyx
    [ 5/55] Cythonizing sklearn/cluster/_k_means_common.pyx
    [ 6/55] Cythonizing sklearn/cluster/_k_means_elkan.pyx
    [ 7/55] Cythonizing sklearn/cluster/_k_means_lloyd.pyx
    [ 8/55] Cythonizing sklearn/cluster/_k_means_minibatch.pyx
    [ 9/55] Cythonizing sklearn/datasets/_svmlight_format_fast.pyx
   [10/55] Cythonizing sklearn/decomposition/_cdnmf_fast.pyx
   [11/55] Cythonizing sklearn/decomposition/_online_lda_fast.pyx
   [12/55] Cythonizing sklearn/ensemble/_gradient_boosting.pyx
```

[13/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/\_binning.pyx  
 [14/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/\_bitset.pyx  
 [15/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/\_gradient\_boosting.pyx  
 [16/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/\_loss.pyx  
 [17/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/\_predictor.pyx  
 [18/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/common.pyx  
 [19/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/histogram.pyx  
 [20/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/splitting.pyx  
 [21/55] Cythonizing  
 sklearn/ensemble/\_hist\_gradient\_boosting/utils.pyx  
 [22/55] Cythonizing sklearn/feature\_extraction/\_hashing\_fast.pyx  
 [23/55] Cythonizing sklearn/linear\_model/\_cd\_fast.pyx  
 [24/55] Cythonizing sklearn/linear\_model/\_sag\_fast.pyx  
 [25/55] Cythonizing sklearn/linear\_model/\_sgd\_fast.pyx  
 [26/55] Cythonizing sklearn/manifold/\_barnes\_hut\_tsne.pyx  
 [27/55] Cythonizing sklearn/manifold/\_utils.pyx  
 [28/55] Cythonizing sklearn/metrics/\_dist\_metrics.pyx  
 [29/55] Cythonizing sklearn/metrics/\_pairwise\_fast.pyx  
 [30/55] Cythonizing  
 sklearn/metrics/cluster/\_expected\_mutual\_info\_fast.pyx  
 [31/55] Cythonizing sklearn/neighbors/\_ball\_tree.pyx  
 [32/55] Cythonizing sklearn/neighbors/\_kd\_tree.pyx  
 [33/55] Cythonizing sklearn/neighbors/\_partition\_nodes.pyx  
 [34/55] Cythonizing sklearn/neighbors/\_quad\_tree.pyx  
 [35/55] Cythonizing  
 sklearn/preprocessing/\_csr\_polynomial\_expansion.pyx  
 [36/55] Cythonizing sklearn/svm/\_liblinear.pyx  
 [37/55] Cythonizing sklearn/svm/\_libsvm.pyx  
 [38/55] Cythonizing sklearn/svm/\_libsvm\_sparse.pyx  
 [39/55] Cythonizing sklearn/svm/\_newrand.pyx  
 [40/55] Cythonizing sklearn/tree/\_criterion.pyx  
 [41/55] Cythonizing sklearn/tree/\_splitter.pyx  
 [42/55] Cythonizing sklearn/tree/\_tree.pyx  
 [43/55] Cythonizing sklearn/tree/\_utils.pyx  
 [44/55] Cythonizing sklearn/utils/\_cython\_blas.pyx  
 [45/55] Cythonizing sklearn/utils/\_fast\_dict.pyx  
 [46/55] Cythonizing sklearn/utils/\_logistic\_sigmoid.pyx  
 [47/55] Cythonizing sklearn/utils/\_openmp\_helpers.pyx  
 [48/55] Cythonizing sklearn/utils/\_random.pyx  
 [49/55] Cythonizing sklearn/utils/\_readonly\_array\_wrapper.pyx

```

[50/55] Cythonizing sklearn/utils/_seq_dataset.pyx
[51/55] Cythonizing sklearn/utils/_typedefs.pyx
[52/55] Cythonizing sklearn/utils/_weight_vector.pyx
[53/55] Cythonizing sklearn/utils/arrayfuncs.pyx
[54/55] Cythonizing sklearn/utils/murmurhash.pyx
[55/55] Cythonizing sklearn/utils/sparsefuncs_fast.pyx
    main()
    File "/Users/monicafam/anaconda3/lib/python3.11/site-
packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 335, in
main
    json_out['return_val'] = hook(**hook_input['kwargs'])
                                ~~~~~~

    File "/Users/monicafam/anaconda3/lib/python3.11/site-
packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 149, in
prepare_metadata_for_build_wheel
    return hook(metadata_directory, config_settings)
            ~~~~~~

    File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/setuptools/build_meta.py",
line 174, in prepare_metadata_for_build_wheel
    self.run_setup()
    File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/setuptools/build_meta.py",
line 268, in run_setup
    self).run_setup(setup_script=setup_script)
            ~~~~~~

    File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/setuptools/build_meta.py",
line 158, in run_setup
    exec(compile(code, __file__, 'exec'), locals())
    File "setup.py", line 319, in <module>
    setup_package()
    File "setup.py", line 315, in setup_package
    setup(**metadata)
    File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/numpy/distutils/core.py", line
135, in setup
    config = configuration()
            ~~~~~~

    File "setup.py", line 201, in configuration
    config.add_subpackage("sklearn")
    File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/numpy/distutils/misc_util.py",

```

```

line 1050, in add_subpackage
    config_list = self.get_subpackage(subpackage_name,
subpackage_path,

~~~~~

File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/numpy/distutils/misc_util.py",
line 1016, in get_subpackage
    config = self._get_configuration_from_setup_py(
    ~~~~~

File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/numpy/distutils/misc_util.py",
line 958, in _get_configuration_from_setup_py
    config = setup_module.configuration(*args)
    ~~~~~

File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-
install-z51vvxl8/scikit-
learn_25af3e2111f84aedb79a9332bdaff2c8/sklearn/setup.py", line 85, in
configuration
    cythonize_extensions(top_path, config)
File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-
install-z51vvxl8/scikit-
learn_25af3e2111f84aedb79a9332bdaff2c8/sklearn/_build_utils/__init__.py", line
78, in cythonize_extensions
    config.ext_modules = cythonize(
    ~~~~~

File
"/private/var/folders/lc/7hp2qk7j3z9g0wz3qxxh6l27h0000gn/T/pip-build-
env-q05065yz/overlay/lib/python3.11/site-packages/Cython/Build/Dependencies.py",
line 1145, in cythonize
    result.get(99999) # seconds
    ~~~~~

File
"/Users/monicafam/anaconda3/lib/python3.11/multiprocessing/pool.py", line 774,
in get
    raise self._value
    Cython.Compiler.Errors.CompileError:
sklearn/ensemble/_hist_gradient_boosting/splitting.pyx
[end of output]

```

**note:** This error originates from a subprocess, and is likely not a problem with pip.

**error:** metadata-generation-failed

× Encountered error while generating package metadata.  
> See above for output.

**note:** This is an issue with the package mentioned above, not pip.  
**hint:** See above for details.

```
[1]: import sklearn
import imblearn

print("scikit-learn version:", sklearn.__version__)
print("imbalanced-learn version:", imblearn.__version__)
```

```
scikit-learn version: 1.4.1.post1
imbalanced-learn version: 0.12.0
```

```
[ ]:
```